

Certified Correlation Clustering at Scale: Distance-Two Relaxations, Anytime Dual Bounds, and Exact Partition Crossover

Quang-Vinh Dang
British University Vietnam, Hung Yen, Vietnam
vinh.dq4@buv.edu.vn ORCID: 0000-0002-3877-8024

Abstract

Correlation clustering (equivalently, cluster editing) has approximation algorithms with ratios below 1.5, but the heuristics used in practice come with no quality guarantee on large graphs: exact solvers stop at a few hundred vertices, and the lower bounds that scale certify only a factor of about two. We attack this gap from both sides. (i) Since optimal clusters have diameter at most two, the metric LP can be restricted to the $O(\sum_v \deg(v)^2)$ pairs at distance at most two without losing validity or LP-rounding guarantees; we strengthen it with star and local subgraph inequalities. (ii) Dual block-coordinate ascent turns this relaxation into an anytime lower bound that is valid after every step. Its certificates are checked in exact arithmetic by a short checker that shares no code with the solver. (iii) The optimality gap decomposes exactly into non-negative local terms, which yields local optimality certificates and guides a large-neighbourhood search. The resulting algorithm, CertiFlip, has expected cost at most 3OPT and returns a certified bound with every solution. On the PACE 2021 exact track its bound proves optimality on 111 of 173 solved instances, against 79 for the root bounds of the state-of-the-art branch-and-bound, and improves the published lower bounds of two open instances. On SNAP graphs with up to 10^6 edges it certifies the best known solutions within 0.25% to 10.1%, where scalable bounds certify factors of 1.17 to 1.81. (iv) Two clusterings can be recombined exactly, block by block of their overlap. The memetic solver PXMmem built on this recombination keeps the guarantee of 3 in expectation. In 150 runs of 600s on fifteen SNAP graphs, run one after the other on the same core as the PACE 2021 winner KaPoCE, it wins 67, ties 49 and loses 34. (v) The main combinatorial arguments are machine-checked in Lean 4.

1 Introduction

In *correlation clustering* [4] we are given a complete graph whose pairs are labelled *similar* (+) or *dissimilar* (−) and look for a partition of the vertices minimising the number of disagreements: + pairs between clusters plus − pairs inside clusters. The number of clusters is not prescribed. Equivalently, the input is the positive graph $G^+ = (V, E^+)$ and the task is to add or delete the fewest edges so that G^+ becomes a disjoint union of cliques (*cluster editing*). The problem is APX-hard [19] and arises in entity resolution, community detection and the aggregation of inconsistent information.

The state of the art is split. On the theory side, the approximation ratio for complete graphs dropped from 3 for the Pivot algorithm [2] to 2.06 by metric-LP rounding [20], below 2 with Sherali–Adams hierarchies [23], to 1.73 with preclustering [24] and to 1.485 with the cluster LP [16]. A preprint from July 2026 reports a deterministic 1.3865-approximation and an upper bound of 1.3865

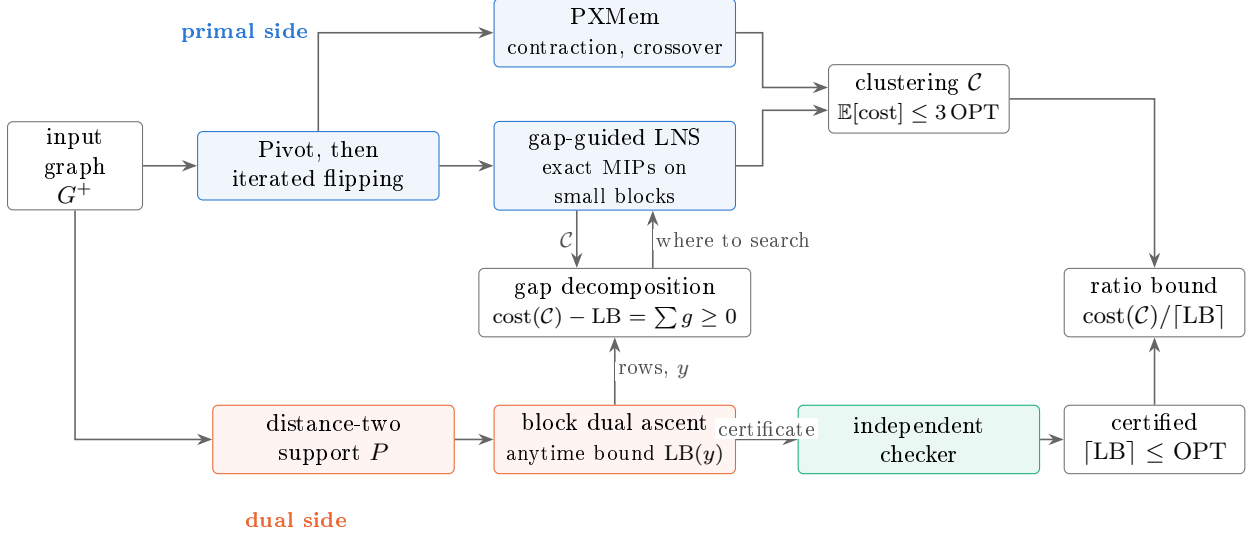


Figure 1: Overview. The primal side (blue) computes clusterings: Pivot and iterated flipping give a start with $\mathbb{E}[\text{cost}] \leq 3 \text{OPT}$, and either the gap-guided LNS of CertiFlip or the memetic search PXMEm improves it. The dual side (orange) builds the distance-two support and a lower bound by block dual ascent. The gap decomposition splits $\text{cost}(\mathcal{C}) - \text{LB}$ into non-negative local terms and tells the LNS where to search. The dual certificate is re-checked by an independent program in exact arithmetic.

on the integrality gap of the cluster LP [35]. The best hardness is $24/23 - \varepsilon$ unless $\text{P} = \text{BPP}$ [16]. Solving the cluster LP takes $2^{\text{poly}(1/\varepsilon)}$ factors even in the sublinear algorithm of Cao et al. [17]. The best *combinatorial* ratio is $2 - 2/13$, achieved by a local search with “flips” in near-linear time [25]. As far as we could determine, none of the sub-3 near-linear algorithms has a public implementation or has been evaluated experimentally.

On the practical side, engineered heuristics are very strong. The winner of the heuristic track of the PACE 2021 challenge, KaPoCE [9], and multilevel memetic solvers [40] find excellent solutions. However, nobody can say *how* good these solutions are on large graphs. Exact branch-and-bound solves instances with a few hundred vertices [10]. Metric-LP solvers based on projection methods reach about 10^4 vertices [57, 67]. The branch-and-bound algorithm of Bläsius et al. [10] uses strong packing lower bounds, but its implementation stores the instance as a dense matrix. The combinatorial bounds that scale to large graphs certify solutions only within a factor of about two [65]. The PACE report [44] lists 27 exact-track instances with up to 620 vertices whose optimum is still unknown.

This paper. We ask for a *practical algorithm with a provable guarantee that also certifies its own quality on the instance at hand*, in the sense of certifying algorithms [52]. Figure 1 shows how the parts fit together. Our contributions are as follows.

1. **Distance-two relaxations** (Section 3.1). Optimal clusters have diameter at most two in G^+ (Lemma 1). We show that the metric LP restricted to pairs at distance at most two, with all pairs at distance at least three fixed to “separated” and the triangle inequalities through them kept, is still a relaxation. Moreover, every feasible point extends to a feasible point of the full metric LP with the same cost (Theorem 3). Hence the 2.5- and 2.06-approximate LP-rounding algorithms [2, 20] keep their guarantees, and the relaxation has $O(\sum_v \deg(v)^2)$

variables instead of $\binom{n}{2}$. We strengthen it with star inequalities and *local subgraph inequalities* (Proposition 4). On real graphs these raise the bound substantially. On **ca-GrQc**, the gap between the metric LP and the best known solution is 23%; with these inequalities it falls below 0.2%.

2. **Anytime dual bounds** (Section 3.2). We compute a lower bound by dual block-coordinate ascent. Each block is a vertex set whose LP is solved exactly by cutting planes, with costs equal to the current reduced costs. The bound is valid after every step, never decreases, and stores only the reduced cost of each pair and the rows with positive multipliers (Theorem 5). Any packing of induced stars or bad triangles is a feasible starting point. We compute one on the sparse support with a ruin-and-recreate local search, using $O(|P|)$ instead of $\Theta(n^2)$ memory. On **ca-HepTh**, after 46 s it is within 0.02% of the root bound of the KaPoCE branch-and-bound, which needs 277 s; together with the block LPs, our bound exceeds it. Blocks come from randomised graph partitions and from regions where the current optimality gap is large.
3. **Gap decomposition and local certificates** (Section 3.3). For any clustering, the difference between its cost and the dual bound splits exactly into non-negative terms attached to pairs and constraints (Theorem 6). As a consequence, re-optimising a vertex set B can improve the cost by at most the gap mass touching B (Corollary 7). This gives a principled way to choose neighbourhoods for a large-neighbourhood search with exact sub-MIPs, and to skip neighbourhoods that are certified optimal.
4. **CertiFlip** (Section 4.1). We implement a heuristic version of the iterated flipping local search of Cohen-Addad et al. [25], with cluster-insertion moves generated by neighbourhood cleaning and multilevel refinement. It explores only a polynomial family of insertions and does not inherit the guarantees of that algorithm. Together with the three components above this yields an algorithm whose expected cost is at most 3 OPT , and at most 2.06 OPT when a single-block relaxation is solved to optimality. It returns a certified lower bound with every solution (Theorem 8).
5. **Exact recombination and PXMmem** (Section 4.2). The blocks of the overlap of two clusterings are independent: taking the better parent in each block gives a clustering that is never worse than either parent (Theorem 11). Together with the contraction of critical cliques (Lemma 10) this yields PXMmem, a memetic solver that keeps the guarantee of 3 in expectation (Theorem 12). Against KaPoCE, run one after the other on the same core with 600 s each, it is better on four of fifteen SNAP graphs and worse on one by the sign test (one each after Holm correction), and equal or indistinguishable on the others.
6. **Machine-checked proofs and checked certificates** (Section 4.3). The main combinatorial lemmas, the incremental cost formulas of the implementation, the validity of the dual bound and the crossover theorem are proved in Lean 4 with Mathlib; Table 1 lists what is and is not covered. Every lower bound reported in the experiments is recomputed from its exported certificate by an independent checker in exact arithmetic.
7. **Evaluation pipeline and experiments** (Section 5). We release an open-source pipeline with ten solvers and five lower bounds. On the 200 PACE 2021 exact-track instances, whose optima are mostly known, the CertiFlip bound proves optimality on 111 of the 173 solved instances. The root bounds of the state-of-the-art branch-and-bound prove it on 79. The CertiFlip bound also improves the published lower bounds of two open instances. We also use

fifteen SNAP graphs with up to $1.1 \cdot 10^6$ vertices. On the eleven whose distance-two support fits into memory (up to $3 \cdot 10^5$ vertices and 10^6 edges), the CertiFlip bound certifies the best known solutions within 0.25% to 10.1% of optimal. Previously scalable bounds certified only factors of 1.17 to 1.81 on the same graphs.

2 Preliminaries and related work

Let $G^+ = (V, E^+)$ with $n = |V|$ and $m = |E^+|$; every pair not in E^+ is negative. For a clustering $\mathcal{C}$ let $x_{uv}^{\mathcal{C}} = 1$ if u, v are in different clusters and 0 otherwise. Then

$$\text{cost}(\mathcal{C}) = \sum_{uv \in E^+} x_{uv}^{\mathcal{C}} + \sum_{uv \notin E^+} (1 - x_{uv}^{\mathcal{C}}).$$

$N(v)$ denotes the positive neighbourhood and $d(u, v)$ the distance in G^+ . The *metric LP* [2, 19] relaxes x to $[0, 1]$ subject to the triangle inequalities $x_{uv} \leq x_{uw} + x_{wv}$ for all triples. A *bad triangle* is a triple u, w, v with $uw, wv \in E^+$ and $uv \notin E^+$; every clustering makes at least one mistake on it.

Approximation algorithms. Pivot repeatedly picks a uniformly random unclustered vertex and clusters it with its unclustered positive neighbours. Its expected cost is at most three times the value of the fractional bad-triangle packing LP [2]. LP-based pivoting joins v to the pivot u with probability $1 - f(x_{uv})$. The choice $f(x) = x$ gives 2.5 [2]. Chawla et al. [20] use $f^-(x) = x$ on negative pairs and, on positive pairs, $f^+(x) = 0$ for $x < a$, $((x - a)/(b - a))^2$ for $a \leq x < b$ and 1 for $x \geq b$, with $a = 0.19$, $b = 0.5095$. Both guarantees hold against the LP value of *any* feasible metric x , a fact we use below. For the history of ratios, parallel, streaming and dynamic algorithms and weighted variants see [6, 17, 22, 25, 28, 35]. Relevant here is the local search of Cohen-Addad et al. [25]. Its only move is a *cluster insertion* $\mathcal{C} + S$: the vertices of S leave their clusters and form a new cluster. A clustering that no insertion improves is a 2-approximation. Increasing the weight of the positive pairs cut by the current solution (a *flip*), re-optimising, and combining the last three solutions with a 3-way pivot yields $2 - 2/13$.

Exact and heuristic solvers. Cluster editing admits linear kernels [18, 39] and practical branch-and-bound algorithms [10, 14]. The latter use packings of stars and induced paths as lower bounds, and mark pairs at distance at least three as forbidden. Bastos et al. [5] note that such pairs are separated in every optimal solution. Twin (critical-clique) reductions and their weighted variants are standard in the parameterised algorithms [12, 13, 21, 39, 56] and in practical tools such as FORCE and TransClust [70, 71]. Cutting-plane methods for the clique partitioning polytope go back to Grötschel and Wakabayashi [37, 38], with recent work on facets and redundancy [42, 46]. Miyauchi and Sukegawa [53] and Miyauchi et al. [54] show that many triangle inequalities of the clique partitioning formulation are redundant; the distance-two relaxation is a different reduction, which removes variables rather than rows. The LP relaxations of Veldt [65], based on strong triadic closure, are also supported on the open wedges of G^+ ; they yield fast approximation algorithms and bounds within a factor of about two, while our relaxation keeps the metric LP on P . For sparse weighted graphs (multicut), dual decomposition and message passing give fast lower bounds [1, 48, 60]. On complete graphs, however, the number of variables is quadratic. Our distance-two support removes exactly this obstacle. Local search heuristics include vertex moves [29], heuristics for clique partitioning [15, 51, 73], local search for multicut [50], Louvain and Leiden with the constant Potts

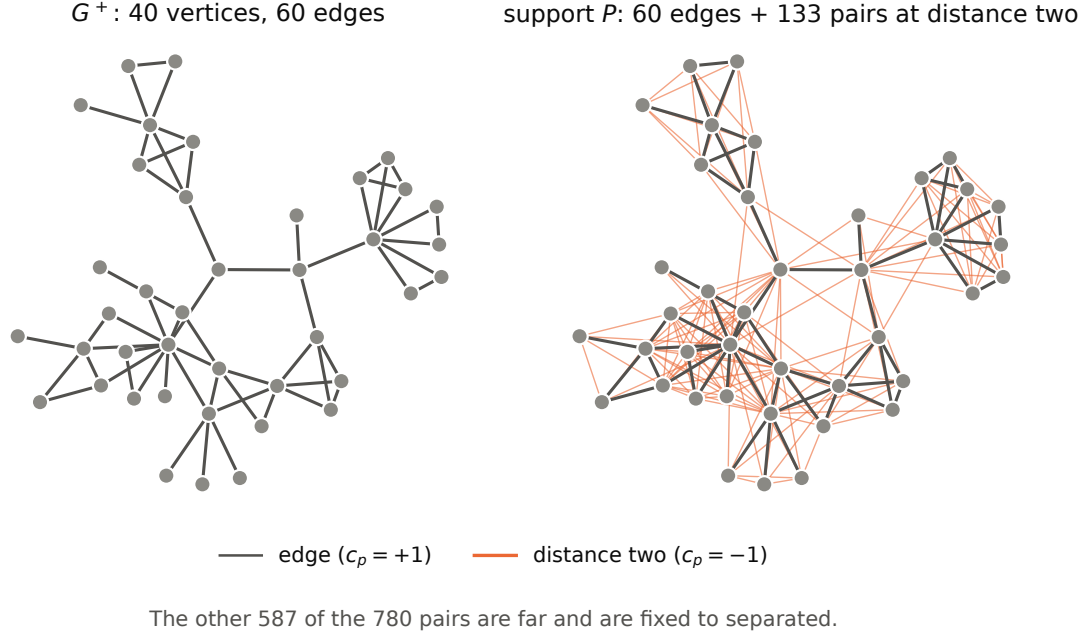


Figure 2: The distance-two support on a 40-vertex neighbourhood of **ca-GrQc**. Left: the positive graph. Right: the support P adds the non-adjacent pairs with a common neighbour (orange); every other pair is far, separated in every optimal clustering (Lemma 1), and carries no variable.

model at resolution $1/2$, which optimises the same objective [11, 64, 66], and memetic multilevel schemes [9, 40].

3 Certified lower bounds

This section develops the lower-bound side of Figure 1: a relaxation supported on the pairs at distance at most two (Section 3.1), an anytime dual bound on it (Section 3.2), and an exact decomposition of the optimality gap that localises where a clustering can still improve (Section 3.3).

3.1 Distance-two relaxations

Lemma 1. *Let $\mathcal{C}$ be an optimal clustering, $S \in \mathcal{C}$ and $v \in S$. Then $|N(v) \cap S| \geq (|S| - 1)/2$. Consequently any two vertices of S are adjacent or have a common neighbour in S . In particular every pair at distance at least three is separated.*

Proof. Moving v to a singleton changes the cost by $|N(v) \cap S| - (|S| - 1 - |N(v) \cap S|)$, which must be non-negative. For non-adjacent $u, v \in S$ the sets $N(u) \cap S$ and $N(v) \cap S$ lie in $S \setminus \{u, v\}$, which has $|S| - 2$ elements, and have total size at least $|S| - 1$, so they intersect. $\square$

Let $N_2 = \{uv \notin E^+ : N(u) \cap N(v) \neq \emptyset\}$, $P = E^+ \cup N_2$ and $F = \binom{V}{2} \setminus P$ (the *far* pairs). Note $|P| \leq m + \sum_w \binom{\deg(w)}{2}$. Figure 2 shows the support on a small neighbourhood of a collaboration network: already there, 587 of the 780 pairs (75%) are far and disappear from the relaxation.

Definition 2 (distance-two relaxation). LP_P has a variable $x_e \in [0, 1]$ for every $e \in P$ and minimises $\sum_{e \in E^+} x_e + \sum_{e \in N_2} (1 - x_e)$ subject to

$$x_{uv} \leq x_{uw} + x_{wv} \quad \text{for all } u, v, w \text{ with } uv, uw, wv \in P, \quad (1)$$

$$x_{uw} + x_{wv} \geq 1 \quad \text{for all } u, v, w \text{ with } uw, wv \in P, \text{ } uv \in F. \quad (2)$$

Theorem 3. Let LP_{std} denote the value of the metric LP.

- (a) $\text{LP}_{\text{std}} \leq \text{LP}_P \leq \text{OPT}$.
- (b) If x is feasible for LP_P , then $\bar{x}$, defined as x on P and 1 on F , is feasible for the metric LP and has the same cost.
- (c) Let $\mathcal{A}$ be a randomised rounding algorithm with $\mathbb{E}[\text{cost}(\mathcal{A}(\bar{x}))] \leq \alpha \text{cost}(\bar{x})$ for every feasible metric $\bar{x}$. Applied to an optimal solution of LP_P it is an α -approximation. This holds in particular for $\alpha = 2.5$ (ACN) and $\alpha = 2.06$ (CMSY). With pivot-based rounding, only pairs in P can be joined, so the rounding takes $O(n + |P|)$ time.

Proof. (b) Consider a triple u, v, w and the number of far pairs among uv, uw, wv . With none, all three triangle inequalities are rows (1). With exactly one, say uv , the inequality $\bar{x}_{uv} = 1 \leq x_{uw} + x_{wv}$ is row (2). The other two inequalities, such as $x_{uw} \leq 1 + x_{wv}$, hold because $x \leq 1$. With two or three far pairs, each inequality has a right-hand side of at least 1. Far pairs are negative and $\bar{x} = 1$ on them, so they contribute 0 to the cost. (a) The first inequality follows from (b). For the second, by Lemma 1 an optimal clustering separates all far pairs. Its cut vector restricted to P therefore satisfies (1) and (2), because it is a metric with $x = 1$ on F , and it has cost OPT . (c) follows from (a) and (b): the expected cost is at most $\alpha \text{cost}(\bar{x}) = \alpha \text{LP}_P \leq \alpha \text{OPT}$. Pivot-based rounding joins v to the pivot u with probability $1 - f(x_{uv})$, and $f(1) = 1$ for both rounding functions. $\square$

The rows (1)–(2) are generated lazily. A violated row needs $x_{uw} + x_{wv} < 1$, so separation around a vertex w only has to look at pairs of “close” neighbours in P , which keeps it fast in practice.

Stronger inequalities. The integrality gap of the metric LP is at least 2 (stars attain it asymptotically) and at most 2.06 [20]. We add two families that are valid for every clustering separating all far pairs, and hence for every optimal clustering.

Proposition 4. Let $\mathcal{C}$ be a clustering in which no far pair shares a cluster.

- (i) (star inequalities; the case $|S| = 1$ of the 2-partition inequalities of Grötschel and Wakabayashi [37]) For every v and $T \subseteq V \setminus \{v\}$, with $y = 1 - x^{\mathcal{C}}$,

$$\sum_{t \in T} y_{vt} - \sum_{\{t, t'\} \subseteq T, \text{ } tt' \in P} y_{tt'} \leq 1.$$

- (ii) (local subgraph inequalities) For every $H \subseteq V$,

$$\sum_{e \in E^+(H)} x_e^{\mathcal{C}} + \sum_{e \in N_2(H)} (1 - x_e^{\mathcal{C}}) \geq \text{OPT}(H),$$

where $\text{OPT}(H)$ is the optimum of the instance induced by H and $E^+(H)$, $N_2(H)$ are the pairs of E^+ and N_2 inside H .

Proof. (i) If the cluster of v contains k vertices of T , the first sum is k and the second is at least $\binom{k}{2}$. Pairs inside T that are not in P are far, so they are separated and contribute 0 to the second sum. Finally $k - \binom{k}{2} \leq 1$. (ii) Restricted to H , $\mathcal{C}$ is a clustering of H . Its mistakes inside H number at least $\text{OPT}(H)$. The far pairs inside H are separated and cost nothing, so all its mistakes are counted on the left-hand side. $\square$

The local subgraph inequalities are local cuts in the sense of Applegate et al. [3]: valid inequalities obtained by solving a small subproblem exactly. Bad triangles ($\text{OPT} = 1$) and induced stars with k independent leaves ($\text{OPT} = k - 1$) are local subgraph inequalities. So are chordless cycles C_k , for which $\text{OPT}(C_k) = \lceil k/2 \rceil$ while the metric LP only certifies $k/2$ (put $x = 1/2$ on the cycle edges). We separate stars greedily around each vertex. For (ii) we grow H ($|H| \leq 8$) by breadth-first search along fractional pairs and compute $\text{OPT}(H)$ by enumerating the at most 4140 set partitions of H .

3.2 Anytime dual bounds

Write the relaxation as $\min\{K + c^\top x : Ax \geq b, 0 \leq x \leq 1\}$ over $x \in \mathbb{R}^P$, with $c_e = 1$ on E^+ , $c_e = -1$ on N_2 and $K = |N_2|$. The rows of (A, b) are any valid inequalities from Section 3.1.

Theorem 5. *For every $y \geq 0$ let $r = c - A^\top y$ and*

$$\text{LB}(y) = K + b^\top y + \sum_{e \in P} \min\{0, r_e\}. \quad (3)$$

(a) $\text{LB}(y) \leq \text{OPT}$.

(b) *Let $B \subseteq V$, let R_B be a set of rows supported on pairs with both endpoints in B , and let $\bar{r}$ be the reduced costs after removing the contribution of the current multipliers on R_B . Replace these multipliers by an optimal dual solution of $\min\{\bar{r}^\top x_B : A_B x_B \geq b_B, 0 \leq x_B \leq 1\}$. This does not decrease LB .*

(c) *If $B = V$ and R_B contains all rows of LP_P , one block step gives $\text{LB} = \text{LP}_P$.*

Proof. (a) Let $\mathcal{C}$ be optimal. By Lemma 1 it separates far pairs, so $\text{OPT} = K + c^\top x^{\mathcal{C}}$ with $x^{\mathcal{C}} \in \{0, 1\}^P$ satisfying $Ax^{\mathcal{C}} \geq b$. Then $\text{OPT} = K + r^\top x^{\mathcal{C}} + y^\top Ax^{\mathcal{C}} \geq K + \sum_e \min\{0, r_e\} + y^\top b$. (b) By LP duality the block LP equals $\max_{y_B \geq 0} b_B^\top y_B + \sum_{e \in P_B} \min\{0, (\bar{r} - A_B^\top y_B)_e\}$. The terms of (3) outside the block do not change, and the old multipliers are feasible for this maximisation. (c) is LP duality for LP_P with the box constraints. $\square$

Part (b) needs an optimal block dual. When a block LP stops at its time limit, its duals are only feasible; the bound computed with them is still valid by part (a), but it may be smaller than before. The implementation therefore evaluates the new bound and keeps a block step only if the bound does not decrease, so the bound is monotone in every case.

Warm start by star packings. Every pair-disjoint family of induced stars (a centre v with $k \geq 2$ pairwise non-adjacent positive neighbours) is a feasible dual solution: put $y = 1$ on the corresponding local subgraph inequalities of Proposition 4(ii), which in x -form read $\sum_t x_{vt} - \sum_{t' \neq v} x_{t't} \geq (k-1) - \binom{k}{2}$. Then $\text{LB}(y)$ equals the packing value $\sum (k-1)$. Indeed, a centre pair has $c_e = 1$ and coefficient 1, and a leaf pair has $c_e = -1$ and coefficient -1 , so every packed pair has reduced cost 0. Unpacked pairs keep $r_e = c_e$. Hence $\text{LB} = K + \sum_S ((k_S - 1) - \binom{k_S}{2}) - |N_2 \setminus \text{packed}|$, and $K - |N_2 \setminus \text{packed}| = \sum_S \binom{k_S}{2}$. Fractional packings of bad triangles, as computed by the Garg-Könemann scheme [33, 36], are installed in the same way on the rows $x_{uw} + x_{wv} - x_{uv} \geq 0$. Packings are the bounds used inside

the KaPoCE branch-and-bound algorithm [10]. Its implementation stores the instance as a dense matrix, so it applies only to graphs with at most about 10^4 vertices. We build a greedy packing on the sparse support, processing centres by decreasing degree, and improve it by *ruin and recreate*. (On dense instances large stars waste pairs, since a star with k leaves uses $k + \binom{k}{2}$ pairs for a value of $k - 1$. There we also try a packing that first packs bad triangles maximally and then extends stars on free pairs, as well as the fractional triangle packing, and install the best of the candidates.) Each step picks a random vertex, removes all stars centred in a random connected region of about 12 vertices around it, and repacks the freed pairs greedily in a random order. A step is kept iff the value does not decrease. The packing is installed as the initial multipliers. Subsequent block steps release the packed rows that lie inside a block and re-optimize them fractionally together with newly separated rows. By Theorem 5(b), the final bound is never below the packing value.

The bound (3) needs only the vector r and the scalar $b^\top y$. We store the rows with positive multipliers so that a later block containing them can re-optimize them. Blocks of the same partition are independent. In a *sweep* we compute a partition into parts of about 2500 vertices with METIS [43] on a randomly relabelled copy of G^+ , so that successive sweeps cut different pairs. Each part is solved with the dual simplex method of HiGHS [41], with lazily separated triangle, star and local subgraph rows. After the sweeps stall we switch to *gap-guided blocks*, grown around the vertices with the largest share of the gap defined next. Because (3) is valid for any $y \geq 0$, the bound remains correct even when a block LP hits its time limit. We only clamp the dual values to be non-negative, and reject a block step that would lower the bound.

3.3 Gap decomposition and local certificates

Theorem 6. *Let $y \geq 0$, $r = c - A^\top y$, and let $\mathcal{C}$ be a clustering that separates all far pairs. Then*

$$\text{cost}(\mathcal{C}) - \text{LB}(y) = \sum_{e \in P} \underbrace{(r_e x_e^{\mathcal{C}} - \min\{0, r_e\})}_{g_e \geq 0} + \sum_i y_i \underbrace{(a_i^\top x^{\mathcal{C}} - b_i)}_{s_i \geq 0}. \quad (4)$$

Proof. $\text{cost}(\mathcal{C}) = K + c^\top x^{\mathcal{C}} = K + r^\top x^{\mathcal{C}} + y^\top A x^{\mathcal{C}}$. Subtract (3). Each $g_e \geq 0$ because $x_e \in \{0, 1\}$, and each $s_i \geq 0$ by the validity of row i . $\square$

A pair term vanishes iff the clustering agrees with the sign of the reduced cost. A row term vanishes iff the row is tight or has multiplier 0. Assigning each term to the endpoints of its pairs gives a *gap map* over the vertices.

Corollary 7 (local certificate). *For $B \subseteq V$ let $\Gamma_B(\mathcal{C})$ be the sum of the terms of (4) whose pair, or one of whose row pairs, has an endpoint in B . Let $\mathcal{C}'$ be any clustering that separates far pairs and coincides with $\mathcal{C}$ on all pairs with both endpoints outside B . Then $\text{cost}(\mathcal{C}) - \text{cost}(\mathcal{C}') \leq \Gamma_B(\mathcal{C})$. In particular, if $\Gamma_B(\mathcal{C}) < 1$ then no re-clustering of B improves $\mathcal{C}$.*

Proof. Apply (4) to both clusterings. Terms not touching B coincide, and the terms of $\mathcal{C}'$ are non-negative. Costs are integers. $\square$

Gap-guided LNS. Large-neighbourhood search [59], here with exact sub-MIPs in the spirit of RINS and local branching [26, 32], fixes the clustering outside a vertex set B ($|B| \leq 60$) and re-optimizes B exactly. Let $\mathcal{K}$ be the outside clusters with a positive edge into B . Re-clusterings of B correspond to clusterings of B together with one super node per $K \in \mathcal{K}$, where super nodes must stay apart. The cost is $\sum_{uv} [uv \in E^+] x_{uv} + [uv \notin E^+] (1 - x_{uv})$ for $u, v \in B$, plus $w_{vK} x_{vK} + (|K| - w_{vK})(1 - x_{vK})$ for each $v \in B$, where w_{vK} counts the positive edges between v and K . We

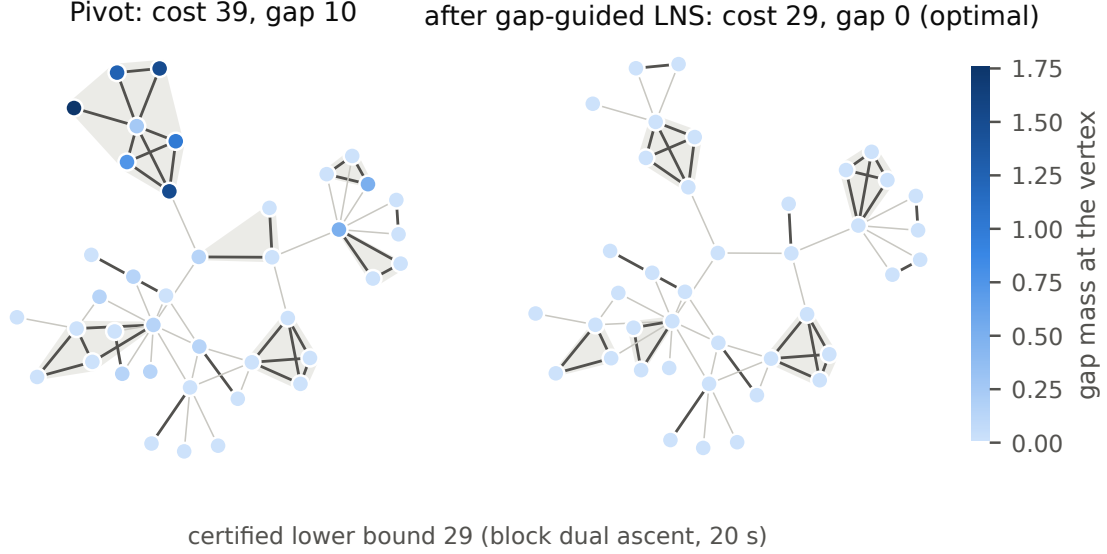


Figure 3: Gap decomposition (Theorem 6) on the neighbourhood of Figure 2. Each vertex is coloured by its share of $\text{cost}(\mathcal{C}) - \text{LB}$ (pair terms split between the endpoints, row terms between the vertices of the row); shaded regions are clusters, dark edges lie inside a cluster. Left: a Pivot clustering. Right: after gap-guided LNS the gap is zero, so the clustering is optimal. Computed by the released code.

solve this MIP with HiGHS and add triangle inequalities lazily until the solution is transitive. To keep Corollary 7 applicable, v may join K only if it is within distance two of every member of K , and far pairs inside B are fixed apart. Neighbourhoods are grown around high-gap vertices, either by breadth-first search or as a union of whole clusters. Neighbourhoods with $\Gamma_B < 1$ are skipped because they are certified optimal. Theorem 6 and Corollary 7 need a clustering that separates far pairs, which the output of the flipping search need not do. Before the search, and after every improvement, we therefore move to a singleton every vertex v with $|N(v) \cap S| < (|S| - 1)/2$ in its cluster S , until there is none. Each such move lowers the cost, and by the argument of Lemma 1 the result separates all far pairs. The certificate $\Gamma_B < 1$ covers re-clusterings of B that also separate far pairs; by Lemma 1 these include every optimal one.

Figure 3 shows the gap map at work on the neighbourhood of Figure 2. For a Pivot clustering the gap of 10 is concentrated on one cluster; the LNS re-optimises the blocks around the high-gap vertices and reaches a clustering whose cost equals the bound, which certifies it optimal.

4 Algorithms

We now describe the two solvers of Figure 1. CertiFlip (Section 4.1) couples a flipping local search with the bound of Section 3 and returns a certificate with every solution. PXMem (Section 4.2) is a memetic solver built on an exact recombination of clusterings; it aims at the best primal value. Section 4.3 states what of both is machine-checked and how the reported bounds are verified.

Algorithm 1 CertiFlip(G^+, T)

```
1:  $\mathcal{C} \leftarrow \text{PIVOT}(G^+)$  with a uniformly random order
2:  $\mathcal{C} \leftarrow \text{ITERATEDFLIP}(\mathcal{C})$  ▷ insertion local search with flips
3: build the distance-two support  $P$ 
4:  $y \leftarrow$  star packing with ruin-and-recreate local search ▷ feasible dual
5:  $y \leftarrow \text{BLOCKASCENT}(P, y, \mathcal{C})$  ▷ METIS sweeps, then gap-guided blocks
6: if one block covered  $V$  and separation converged then
7:    $\mathcal{C} \leftarrow$  best of  $\mathcal{C}$  and  $\text{ITERATEDFLIP}(\text{CMSY-ROUND}(x^*))$  over several roundings
8: end if
9:  $\mathcal{C} \leftarrow \text{GAPLNS}(\mathcal{C}, y)$  until time  $T$ 
10: return  $\mathcal{C}$  and  $\lceil \text{LB}(y) \rceil$  ▷ lines 4–5 share half of the budget  $T$ 
```

4.1 CertiFlip

Insertion local search. For every root r we build one candidate insertion $S(r)$ by *cleaning* $N[r]$. Starting from $S = N[r]$, we repeat a few sweeps over $v \in N[r] \setminus \{r\}$: we put v in S iff joining S costs v less than staying in the remainder of its current cluster. Both costs are computed exactly from counts of weighted positive neighbours. We then compute the exact change of the objective in $O(\sum_{v \in S} \deg v)$ time and apply the insertion if it is negative. Rounds of insertions alternate with a multilevel (Louvain-type) vertex-move local search. That search works on the weighted objective, moves super nodes at coarse levels and hence also merges clusters. ITERATEDFLIP follows Cohen-Addad et al. [25]. Positive weights take values in $\{1, 1.5, 2\}$ (we double all weights to stay in integers). Each round flips the pairs cut by the previous solution, re-optimises, flips and re-optimises again, and combines the last three solutions with their 3-way pivot. The best candidate under the original objective is returned.

Theorem 8. *Let $\mathcal{C}$ and LB be the output of Algorithm 1.*

- (a) $\mathbb{E}[\text{cost}(\mathcal{C})] \leq 3 \text{OPT}$.
- (b) $\text{LB} \leq \text{OPT}$, so $\text{cost}(\mathcal{C})/\lceil \text{LB} \rceil$ is an upper bound on the approximation ratio achieved on the instance.
- (c) If line 7 is executed, $\mathbb{E}[\text{cost}(\mathcal{C})] \leq 2.06 \text{OPT}$.

Proof. Every step after line 1 returns a clustering no worse than its input: the local searches only apply improving moves, ITERATEDFLIP returns the best of its candidates, which include the local optimum reached from its input, and GAPLNS accepts only improvements. (a) then follows from the Pivot guarantee [2]. (b) is Theorem 5(a) and the integrality of OPT. (c) If one block covered V and separation converged, the block LP primal x^* satisfies all rows (1)–(2), so it is feasible for LP_P . Its cost is the value of the block LP. Every row of the block LP is valid for an optimal clustering (Lemma 1 and Proposition 4), whose cut vector is therefore feasible for the block LP; hence the block LP value is at most OPT. By Theorem 3(b,c), $\mathbb{E}[\text{cost}(\text{CMSY-ROUND}(x^*))] \leq 2.06 \text{OPT}$. The remaining steps are monotone. $\square$

Remark 9. Part (a) holds for any procedure that starts from a Pivot clustering and never makes its solution worse; it is a safeguard, not a distinguishing property. If the insertion local search reached an optimum with respect to *all* cluster insertions, Cohen-Addad et al. [25] would give a factor of 2, and $2 - 2/13$ with enough flipping rounds. Our candidate generation explores a polynomial

subfamily of insertions, so we do not claim these factors. The certificate of Theorem 8(b), however, is valid in every case.

Running time. Pivot takes $O(n+m)$ time. One round of insertions takes $O(\sum_v \sum_{u \in N(v)} \deg u) = O(\sum_u \deg(u)^2)$, the same order as the size of P and the number of bad triangles. The support takes $O(\sum_w \deg(w)^2)$ time and memory. The block LPs dominate the running time, and the time spent on them is set by the budget T .

4.2 Exact recombination: the PXMem solver

CertiFlip certifies its solutions, but on large graphs they remain 0.1%–0.4% above those of KaPoCE (Section 5). This subsection describes a second primal solver, *PXMem*. It rests on two elementary exact facts: twins can be contracted, and two clusterings can be recombined block by block without losing anything. Neither fact is deep, and both have close relatives in the literature, which we point out below; what we add is their combination into a solver whose incremental formulas and recombination are machine-checked (Section 4.3).

4.2.1 Critical-clique contraction

Two vertices $u \neq v$ are *twins* if $uv \in E^+$ and $N(u) \setminus \{v\} = N(v) \setminus \{u\}$, i.e. they have the same closed neighbourhood. The classes of twins are the *critical cliques* of Guo [39]. The following lemma is due to Guo [39] (see also Protti et al. [56]); we include the short proof because the implementation relies on its exact form.

Lemma 10 (39). *Let u, v be twins and $\mathcal{C}$ a clustering that separates them. Moving v into the cluster of u and moving u into the cluster of v change the cost by δ_1 and δ_2 with $\delta_1 + \delta_2 = -2$. Hence every optimal clustering keeps every critical clique inside one cluster.*

Proof. Let $A \ni u$ and $B \ni v$ be the two clusters. For a cluster X let $f(X)$ be the number of mistakes on the pairs zw with $w \in V \setminus \{u, v\}$ when a vertex $z \in \{u, v\}$ is placed in X (all other vertices staying where they are). It is the same for u and v because they have the same neighbours in $V \setminus \{u, v\}$. The moves do not change $A \setminus \{u, v\}$ and $B \setminus \{u, v\}$. Besides the cut positive pair uv , the current cost of the pairs involving u or v is $f(A) + f(B)$; after the first move it is $2f(A)$, after the second $2f(B)$. So $\delta_1 + \delta_2 = 2f(A) + 2f(B) - 2(f(A) + f(B) + 1) = -2$ and one of the moves is a strict improvement. $\square$

Contracting every critical clique K to a node of size $s_K = |K|$ gives a weighted instance on the node set U : two nodes $K \neq L$ carry the weight $w_{KL} = s_K s_L$ if K and L are adjacent and 0 otherwise, and the cost of a clustering c of U is

$$\text{cost}_w(c) = \sum_{\{K,L\}: c(K)=c(L)} (s_K s_L - w_{KL}) + \sum_{\{K,L\}: c(K) \neq c(L)} w_{KL}. \quad (5)$$

The pairs inside a critical clique are positive and never cut. Hence the cost of the expanded clustering on G^+ equals $\text{cost}_w(c)$, and by Lemma 10 an optimal clustering of the contracted instance expands to an optimal clustering of G^+ . Merging vertices into weighted nodes in this way is standard in cluster editing [13, 39]. On the collaboration graphs the contraction removes 13%–22% of the vertices. Pivot never separates twins either: whether a vertex joins the cluster of the pivot p depends only on its closed neighbourhood, so each round treats twins alike. A Pivot clustering therefore maps to the contracted instance without changing its cost.

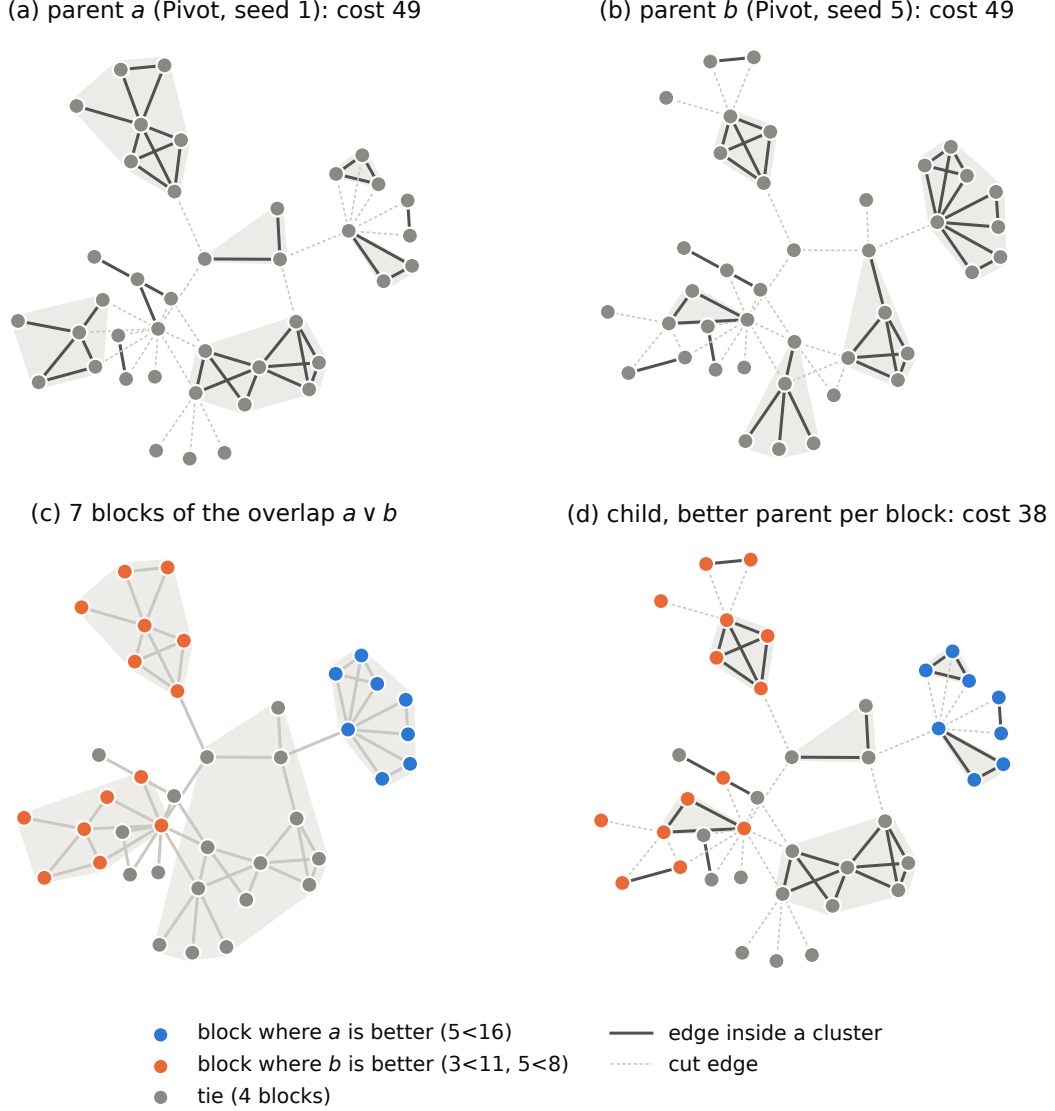


Figure 4: Partition crossover (Theorem 11) computed by the released code. (a), (b): two Pivot clusterings of the neighbourhood of Figure 2, both of cost 49; shaded regions are clusters. (c): the connected components of the overlap graph H split the vertices into seven blocks, and on each block the two parents can be compared on their own. (d): taking the better parent on every block gives a clustering of cost 38, better than both parents.

4.2.2 Partition crossover

Figure 4 illustrates the theorem below on two Pivot clusterings of the neighbourhood of Figure 2.

Theorem 11 (partition crossover). *Let the cost of a clustering be any sum of pair terms that depend only on whether the two vertices of the pair share a cluster (this covers cost and the weighted cost (5)). Let a and b be clusterings and let H be the bipartite graph whose nodes are the clusters of a and of b , with an edge between $A \in a$ and $B \in b$ whenever $A \cap B \neq \emptyset$. Each connected component of H covers a vertex set X , the block, which is a union of clusters of a and also a union of clusters of*

b. For a clustering c let $\text{cost}_X(c)$ count the mistakes on the pairs inside X . Choose, independently in every block, the clusters of b if $\text{cost}_X(b) < \text{cost}_X(a)$ and those of a otherwise. The resulting clustering χ satisfies

$$\text{cost}(\chi) = \text{cost}(a) - \sum_X \max(0, \text{cost}_X(a) - \text{cost}_X(b)) \leq \min(\text{cost}(a), \text{cost}(b)).$$

Proof. A pair u, v in different blocks is separated by a , by b and by χ , because each of their clusters lies inside one block. It therefore contributes the same amount to all three costs. A pair inside a block X is treated by χ exactly as by the parent chosen for X . Summing over the pairs gives the equality. Every term of the sum is non-negative, and exchanging the roles of a and b gives the bound by $\text{cost}(b)$. $\square$

The blocks are the connected components of a bipartite graph with at most n edges, and the block costs need one pass over E^+ . Hence χ is computed in $O(n + m)$ time. The blocks form the join $a \vee b$ of the two partitions in the partition lattice: the finest partition whose parts are unions of clusters of a and also of b . It is the finest partition for which every block-wise choice between the parents yields each parent as one of the choices and leaves the pairs between blocks separated in both. The idea of decomposing two parents into independent components and taking the better parent in each is that of the partition crossover of Whitley et al. [68, 69] for tours, of its generalisation GPX2 [63], and of partition crossover for pseudo-Boolean functions [62]; see also optimal recombination [30]. Recombination through the overlap of the parents' groups is also common in grouping genetic algorithms [31, 34], memetic graph partitioning [7, 58], memetic clustering [8] and ensemble clustering with core groups [55]. The argument is a few lines, and we claim no novelty for it. It combines *any* two clusterings, including clusterings produced by different solvers. On our graphs, two independent runs typically differ in thousands of blocks, most of them ties, and each run wins some of the others.

4.2.3 Annealing moves

The search moves nodes of the contracted instance. Let $S_X = \sum_{K \in X} s_K$ be the size of a cluster X and $w(K, X) = \sum_{L \in X \setminus \{K\}} w_{KL}$. Moving a node K of size s from its cluster A to a cluster B changes cost_w by

$$\delta = s(S_B - S_A + s) + 2(w(K, A) - w(K, B)). \quad (6)$$

Exchanging $K \in A$ with $L \in B$ changes it by the sum of two such terms, where the second one uses $S_A - s_K$, $S_B + s_K$, $w(L, B) + w_{KL}$ and $w(L, A) - w_{KL}$. Both are evaluated in $O(\deg K + \deg L)$ time. The simulated annealing [45] proposes the move to the cluster of a random neighbour, the best move, or a move to a new cluster, and accepts with the Metropolis rule under a geometric cooling schedule. It records the best state with an undo journal, so a new best state costs $O(1)$ instead of a copy of the labelling. This alone made the annealing 7.3 times faster on **com-Amazon**.

Region-wise acceptance. Theorem 11 turns annealing into an iterated local search with a decomposed acceptance test. From the current clustering x we anneal a copy, starting at a raised temperature, down to its final state y , and set $x \leftarrow \chi(x, y)$. Every block in which y improved on x is kept, even if y is worse overall, and the cost of x never increases.

Localized annealing with rollback. A step grows a region of at most 30 nodes by breadth-first search around a centre, anneals only the nodes of the region from a high to a low temperature,

Algorithm 2 PXM_{Mem}(G^+, T)

```
1: contract the critical cliques
2: for  $i = 1, \dots, 4$  do
3:    $p_i \leftarrow \text{PIVOT}(G^+)$ ;  $f_i \leftarrow$  one round of  $\text{ITERATEDFLIP}(p_i)$ 
4:    $x_i \leftarrow$  the cheaper of  $p_i$  and  $f_i$  on the contracted nodes; anneal  $x_i$ 
5: end for
6: while time remains do
7:   choose an operator by an  $\varepsilon$ -greedy bandit on the recent gain per second
8:   crossover: parents  $a, b$  by binary tournaments;  $z \leftarrow \chi(a, b)$ ;
      contract the blocks common to  $a$  and  $b$  and run a multilevel local search from  $z$ ;
      anneal on the nodes where  $a$  and  $b$  disagree; accept region-wise; crowding replacement
9:   PX-annealing step on a member (temperature chosen by a second bandit)
10:  localized annealing with rollback on a member
11: end while
12: return the cheapest member, expanded to  $V$ 
```

finishes with greedy best moves, and undoes all moves if their total change is positive. The undo is exact: moves are reversed in LIFO order together with the free list of cluster labels. Half of the centres are drawn with probability proportional to the degree. Comparing our solutions with those of KaPoCE block by block showed why: the blocks in which KaPoCE was better all contained a vertex of high degree.

4.2.4 The memetic search

Algorithm 2 keeps a population of four clusterings. A crossover child enters the population only by replacing a member that is not cheaper, preferring the most similar members (crowding); the other two operators replace a member only if they improve it. Contracting the blocks common to both parents and continuing with a multilevel search from the recombined clustering follows the recombination of memetic graph partitioners and clusterers [8, 40, 58]. The flipping round of line 3 costs a few full local-search passes whatever its time limit; with budgets below 300s the seeds are therefore Pivot clusterings only, and no further seed is generated once half of the budget is used.

Theorem 12. *Let $\mathcal{C}$ be the output of Algorithm 2 and p_1 the first Pivot clustering. Then $\text{cost}(\mathcal{C}) \leq \text{cost}(p_1)$ for every outcome of the random choices, and hence $\mathbb{E}[\text{cost}(\mathcal{C})] \leq 3 \text{OPT}$. Together with a lower bound LB from Section 3.2, $\text{cost}(\mathcal{C})/\lceil \text{LB} \rceil$ bounds the ratio achieved on the instance.*

The guarantee is inherited from the Pivot seed, as for any procedure that starts from a Pivot clustering and never makes its best solution worse; we state it because every operator of the search must respect it, and this is what the formal development checks.

Proof. The seed x_1 costs at most $\text{cost}(p_1)$: Pivot does not separate twins, so its contraction preserves its cost, and x_1 is the cheaper of the two candidates. The annealing returns the cheapest state of a trajectory that starts at its input; region-wise acceptance does not increase the cost by Theorem 11; the localized annealing keeps a step only if its total change is non-positive; and every replacement is by a clustering that is not more expensive than the member it replaces. By induction over the replacements, some member always costs at most $\text{cost}(x_1)$, so the cheapest final member costs at most $\text{cost}(p_1)$. Taking expectations and applying the Pivot guarantee [2] gives the second claim. The last claim is Theorem 5(a). $\square$

Table 1: Coverage of the formal development. “Full”: the statement of the paper is proved as stated (up to the ordered-pair convention); “partial”: the part named is proved; “reduction”: proved from the cited guarantee taken as a hypothesis; “–”: not formalised.

result	coverage	Lean theorems
Lemma 1	partial	far pairs are separated by every optimal clustering; <code>optimal_separates_far</code> , <code>sepFar_of_optimal</code>
Theorem 3	–	
Proposition 4(i)	full	<code>star_row_valid</code> (star rows in x -form, far-separating clusterings)
Proposition 4(ii)	partial	bad triangles and induced stars: <code>bad_triangle_costIn</code> , <code>star_bound</code> ; packings: <code>packing_bound</code>
rows (1)–(2)	full	<code>sep_triangle</code> , <code>sep_far_triangle</code>
Theorem 5(a)	full	<code>far_dual_bound</code> , <code>far_dual_le_opt</code> (bound on P for rows valid for far-separating clusterings, $\leq$ the cost of every clustering); <code>cost_eq_supp</code>
Theorem 5(b,c), Theorem 6, Corollary 7	–	
Theorem 8	(a) reduction, (b) full, (c) –	(a) as Theorem 12; (b) is Theorem 5(a)
Lemma 10	full	<code>twins_split_improvable</code> , <code>optimal_keeps_twins</code>
Pivot keeps twins together	partial	one round: <code>pivot_round_twins</code>
contraction (5)	partial	cost identity: <code>cost_contract</code>
move and swap formulas (6)	full	<code>cost_move</code> , <code>costW_move</code> , <code>costW_swap</code> , <code>cost_update</code>
Theorem 11	full	any pairwise-additive cost: <code>crossoverP_le</code> ; unweighted: <code>crossover_le</code> ; weighted: <code>crossoverW_le</code> ; relabelling: <code>cost_comp_injective</code> , <code>costW_comp_injective</code>
Theorem 12	reduction	<code>reach_le</code> , <code>output_le</code> , <code>search_guarantee</code> : non-worsening replacements keep a member no worse than the seed; the Pivot guarantee [2] is a hypothesis

4.3 Machine-checked proofs and certificate checking

The correctness of the solvers rests on a number of short combinatorial arguments and on exact formulas that the implementation evaluates millions of times per second. An error in a sign or in a boundary case would not crash the code; it would silently produce worse or invalid results. We therefore formalised part of the mathematics in Lean 4 [27] with Mathlib [61]. The development has about 1,800 lines in fourteen files and is part of the repository (`lean/CCProofs`). Every theorem named in Table 1 is checked by the Lean kernel without `sorry` and depends only on the standard axioms `propext`, `Classical.choice` and `Quot.sound`.

The formal model is the one of Section 2: a finite vertex set, a simple graph G^+ and a labelling $c : V \rightarrow \alpha$ into an arbitrary type with decidable equality. The cost is summed over ordered pairs, so every count appears multiplied by two (`cost_eq_two_mul_card` relates it to the unordered count). The weighted instance has integer node sizes and symmetric weights, as in (5).

Scope. The formalisation covers the mathematics, not the Python and Numba code, and not all results of the paper (Table 1). Three points connect it to the numbers we report. First, the lower bounds are computed in floating point, but they are certified independently: the solver writes out every row with a positive multiplier, and a separate checker (`experiments/check_certificate.py`), which shares no code with the solver, (i) checks that every pair of every row lies in P , (ii) checks the validity of every row for all far-separating clusterings, by enumerating the far-separating partitions of its vertex set for rows on at most nine vertices and by the closed form of `star_row_valid` for star rows, (iii) rounds the multipliers down to multiples of 2^{-30} , which keeps them non-negative, and (iv) evaluates $b^\top y + \sum_p (\min\{0, c_p - (A^\top y)_p\} - \min\{0, c_p\})$ in exact integer arithmetic. This is the left-hand side of `far_dual_bound`, so every bound we report as certified is a proved lower bound on OPT up to the correctness of this short checker, not of the solver. Second, `star_row_valid` and `sep_far_triangle` hold only for clusterings that separate far pairs; they enter the bound through `far_dual_le_opt`, which uses that optimal clusterings do so. Third, the code is tested against the formal statements: exact costs of random clusterings on contracted instances, the tracked cost against a recomputation at the end of annealing runs with many moves, swaps and rollbacks, and brute force on small graphs. These tests check final states, not every step.

5 Experiments

5.1 Setup

Implementation. All our algorithms are implemented in Python with Numba-compiled kernels in the package `ccbench`. LPs and MIPs are solved with HiGHS 1.15 [41] and graph partitions are computed with METIS [43]. Experiments ran on a 4-core virtual machine with 15 GB of memory. Each run used one thread. Four runs executed in parallel on the PACE instances and three on the SNAP graphs.

Algorithms. We compare the following. *Pivot* [2]. *Pivot-50*: the best of 50 Pivot runs. *MatchFlipPivot* [65]. *Vote*: best-improvement vertex moves from singletons, in the spirit of [29]. *Louvain-CC*: our multilevel implementation of the vertex-move local search. *Leiden-CPM*: Leiden with the constant Potts model at resolution 1/2 [64], which optimises the correlation clustering objective [66]. *Insertion LS*: our cluster-insertion local search. *IteratedFlip*: five flipping rounds, following [25]. *KaPoCE*: the PACE 2021 winner of both tracks, heuristic version [9]. *CertiFlip*: Algorithm 1. *PXMem*: Algorithm 2 (Section 5.5). KaPoCE and CertiFlip receive the same wall-clock budget T . The other methods run to completion, with three seeds each.

Lower bounds. We report the following bounds. *Greedy packing*: a maximal pair-disjoint packing of bad triangles, i.e. the matching bound of MatchFlipPivot. *Triangle packing*: a fractional packing by the Garg–Könemann scheme [33, 36] with $\varepsilon = 0.05$, normalised by its maximum load, hence always valid. Its value approximates the covering LP over bad triangles. The *CertiFlip bound*: Theorem 5. On PACE instances we also use the root *star-packing* and *P_3 -packing* bounds of the KaPoCE branch-and-bound, as published by its authors [10]. All reported bounds are rounded up to integers.

Instances. (i) The 200 instances of the PACE 2021 exact track [44] ($n \leq 620$). 173 of them have a known optimum and 27 are open. (ii) Fifteen SNAP graphs, used as complete instances with E^+ given by the edges (for signed networks, by the positive edges): collaboration networks

Table 2: PACE 2021 exact track (200 instances): primal quality. “Hit” counts instances on which the best of the runs matches the best known value; the gap is the relative excess of the mean cost over the best known value.

Algorithm	best known hit	mean gap (%)	max gap (%)	time (s)
Pivot	13/200	65.173	638.98	0.05
Pivot-50	25/200	13.491	32.67	0.04
MatchFlipPivot	3/200	160.902	700.00	2.44
Vote	135/200	1.075	11.78	0.04
Louvain-CC	153/200	0.544	5.56	0.04
Leiden-CPM	158/200	0.449	5.00	0.54
Insertion LS	173/200	0.147	3.02	0.22
IteratedFlip	187/200	0.018	0.45	3.05
KaPoCE	200/200	0.000	0.00	42.32
CertiFlip	191/200	0.007	0.32	33.26

ca-*, email-Enron, loc-Brightkite, soc-Epinions, the positive parts of Slashdot, Epinions and the two Bitcoin trust networks [47, 49], and the community graphs com-Amazon, com-DBLP and com-Youtube [72]. (iii) The 200 instances of the PACE 2021 heuristic track [44] (up to $2.0 \cdot 10^6$ vertices), including the 100 instances that were hidden during the challenge; they were used only for the final comparison with KaPoCE.

5.2 PACE 2021 exact track

Table 2 compares the solutions with the best known value of each instance. That value is the published optimum or upper bound [10], or the best value found in our runs if lower. KaPoCE and CertiFlip had a budget of 60 s. The other methods ran to completion, and we report the mean over three seeds.

Primal quality. Our implementation of the iterated flipping local search matches the best known value on 187 of the 200 instances. Its mean gap is 0.018% and a run takes three seconds on average. Its approximation guarantee is only 3 in expectation, but in practice it is an order of magnitude better than Louvain- or Leiden-type local search. Those methods optimise the same objective but reach only 153 and 158 instances, with mean gaps of 0.54% and 0.45%. CertiFlip reaches 191 instances, and its largest gap is 0.32%. KaPoCE reaches all 200 instances. This is not surprising: the best known values come from its own branch-and-bound runs [10], and it was engineered for this benchmark. Pivot and MatchFlipPivot are far from optimal on these fairly dense instances. Among the 27 open instances, the best published upper bound of **exact183** is 2789, and both KaPoCE and CertiFlip find solutions of cost 2788 within the budget, so this improvement is shared by the two solvers. We verified the CertiFlip solution independently, and it is included in the repository.

Lower bounds. Table 3 and Figure 5 compare the bounds. All CertiFlip bounds in this subsection are the values returned by the independent checker of Section 4.3 on the exported certificates; the checker accepted all 200 certificates. The CertiFlip bound (Theorem 5), computed within half of the 60 s budget, proves optimality on 111 of the 173 instances. The root star-packing bound of the KaPoCE branch-and-bound proves optimality on 79, its P_3 -packing bound on 59, and the classical packings of bad triangles on at most 30. Our bound exceeds the star-packing bound on 60 instances,

Table 3: PACE 2021 exact track: lower bounds on the 173 instances with known optimum. The last two rows are the root bounds of the KaPoCE branch-and-bound as published by its authors. “LB = OPT” counts the instances whose bound proves optimality.

Lower bound	instances	mean LB/OPT	min LB/OPT	LB = OPT
CertiFlip bound	173	0.9884	0.865	111
Triangle packing (MWU)	173	0.9664	0.811	30
Greedy packing	173	0.8612	0.660	14
B&B star packing	173	0.9948	0.891	79
B&B P_3 packing	173	0.9742	0.813	59

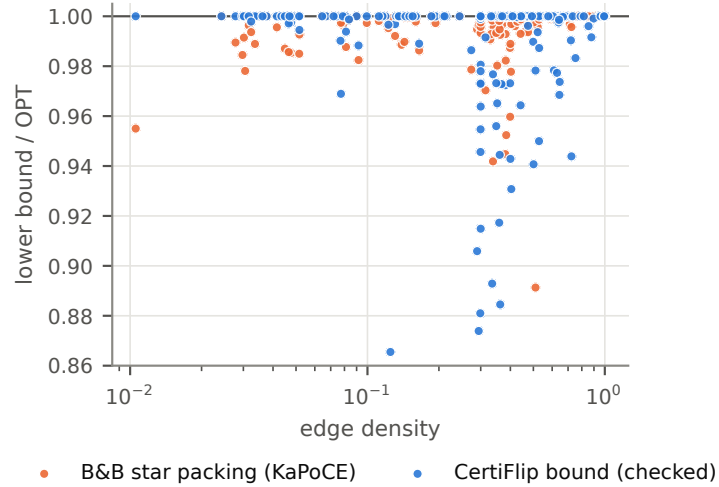


Figure 5: PACE 2021 exact track, 173 instances with known optimum: lower bound divided by the optimum against edge density, for the checked CertiFlip bound and the root star-packing bound of the KaPoCE branch-and-bound.

is lower on 43 and equal on 70. The comparison depends on density. On the 35 instances with edge density below 0.1, our bound attains 99.8% of the optimum on average, against 99.3% for the star packing. On denser instances the star packing is better (99.4% to 99.7% against 98.0% to 98.8%). There the block LPs are too large to converge within the budget, and the bound falls back to the best packing.

Table 4 lists the open instances. Our bound improves the published lower bound on two of them: on `exact179` from 620 to 632, so the optimum is now known to be 632 or 633, and on `exact180` from 1052 to 1068. On the other open instances, which are dense, the published star-packing bound remains stronger.

Table 4: The 27 open PACE 2021 exact instances: published upper bound (UB) and star-packing lower bound (LB) [10], our best solution over all methods and runs, and the checked CertiFlip bound of one 60 s run. Bold marks improvements over the published values.

Instance	n	m	published UB	published LB	our UB	our LB
exact042	100	1563	1055	1006	1055	953
exact043	100	1624	984	968	984	955
exact051	120	2251	1619	1508	1619	1419
exact052	120	2448	1473	1431	1473	1360
exact060	124	2288	1491	1427	1491	1348
exact069	140	2915	2140	1994	2140	1862
exact070	140	3014	1792	1762	1792	1717
exact082	160	4015	2434	2397	2434	2261
exact083	160	4259	2939	2808	2939	2613
exact091	180	4602	3234	3072	3234	2848
exact092	180	6067	3563	3520	3563	3027
exact101	200	6731	4198	4135	4198	3762
exact102	200	6746	3925	3895	3925	3593
exact103	200	7081	4572	4381	4572	3995
exact105	200	12309	5320	5314	5320	5085
exact167	300	14191	9861	9458	9861	8573
exact169	300	15456	9561	9380	9561	7647
exact179	330	1370	633	620	633	632
exact180	330	2256	1081	1052	1081	1068
exact181	330	3253	1563	1513	1563	1504
exact182	330	4329	2159	2091	2159	1945
exact183	330	5613	2789	2717	2788	2566
exact184	330	7270	3684	3582	3684	3371
exact191	400	31201	18427	18288	18427	15264
exact195	620	3992	2529	2450	2529	2363
exact197	620	35188	15831	15782	15831	13849
exact198	620	91572	46764	46717	46764	42743

5.3 SNAP graphs

Table 5 reports the best objective value per method on the SNAP graphs. KaPoCE and CertiFlip had 600s. CertiFlip spends half of its remaining budget on the lower bound. On the four graphs whose distance-two support exceeds $3 \cdot 10^7$ pairs, the support does not fit into memory together with three parallel runs. There CertiFlip skips the bound and spends the whole budget on the primal side. MatchFlipPivot and the packing bounds are not reported on these graphs.

Primal quality. KaPoCE finds the best solution on every graph. Our implementation of the iterated flipping local search stays within 0.10%–0.44% of it. It takes between one second and three minutes, depending on the graph, compared with KaPoCE’s ten minutes. Louvain- and Leiden-type local search, which optimise the same objective, are 0.6% to 2.7% worse. Pivot, the best of 50 Pivot runs and MatchFlipPivot are far from competitive, which is consistent with the PACE results. CertiFlip spends half of its budget on the bound, and its gap-guided LNS improves the flipping solution only slightly on these graphs. Its primal quality is therefore that of IteratedFlip. On four graphs CertiFlip exceeded the budget by up to a factor of two, because single local-search rounds or sub-MIP constructions are not interrupted. This does not change the comparison, since

Table 5: SNAP graphs: best objective value found (bold: best in the row). All methods except KaPoCE and CertiFlip ran to completion; for these, the best of three seeds is shown. “–”: support too large for memory (MatchFlipPivot) or not finished within one hour (com-Youtube).

Graph	n	m	Pivot	Pivot-50	MatchFlipPivot	Vote	Louvain-CC	Leiden-CPM	Insertion LS	IteratedFlip	KaPoCE	CertiFlip
ca-GrQc	5242	14484	9811	8499	13068	6185	6158	6161	6080	6070	6050	6052
ca-HepTh	9877	25973	22573	22573	29385	16255	16190	16193	15929	15877	15814	15857
ca-HepPh	12008	118489	88732	63780	103445	46515	46477	46406	46222	46191	46133	46199
ca-AstroPh	18772	198050	200307	184322	229913	126779	126633	126572	125746	125692	125559	125684
ca-CondMat	23133	93439	87152	80134	110751	57856	57670	57611	57050	56998	56835	56980
email-Enron	36692	183831	192960	185219	214677	151761	151611	151561	150906	150698	150430	150701
loc-Brightkite	58228	214078	245496	236081	257088	178077	177773	177730	175734	175103	174695	175114
soc-Epinions	75879	405740	496495	469475	–	368339	368108	368094	366163	365580	365162	365601
Slashdot+	75144	382882	469502	457004	–	356033	355962	355915	353542	352349	351859	352367
Epinions+	114467	592591	744964	669690	–	525546	525237	525372	522976	522344	521825	522369
BitcoinOTC+	5573	18591	20058	19456	23807	16702	16675	16701	16554	16497	16462	16502
BitcoinAlpha+	3683	12972	14223	13366	16006	11625	11608	11619	11503	11461	11437	11464
com-Amazon	334863	925872	904427	897069	1138129	632915	629054	626887	617758	615008	612313	615002
com-DBLP	317080	1049866	948075	933506	1166731	617025	615183	614162	608743	608173	606752	608212
com-Youtube	1134890	2987624	3638780	3389320	–	2683459	2682697	–	–	–	2649253	2652138

Table 6: SNAP graphs: lower bounds and certified optimality gaps. The gap is $(UB - LB)/LB$, where UB is the best value found in any of our runs (including the head-to-head runs of Section 5.5) and LB is the CertiFlip bound accepted by the independent checker (one 600 s run on a Colab machine, Section 5.5).

Graph	best UB	greedy packing	triangle packing	CertiFlip LB	certified gap (%)
ca-GrQc	6050	4479	4725	6035	0.25
ca-HepTh	15813	10835	10887	15699	0.73
ca-HepPh	46133	35012	39495	43314	6.51
ca-AstroPh	125541	85348	86009	117966	6.42
ca-CondMat	56834	39380	39704	55158	3.04
email-Enron	150419	85082	83057	140551	7.02
loc-Brightkite	174695	102317	100659	163166	7.07
soc-Epinions	365137	–	–	–	–
Slashdot+	351856	–	–	–	–
Epinions+	521786	–	–	–	–
BitcoinOTC+	16462	9137	8774	15583	5.64
BitcoinAlpha+	11437	6335	6134	10722	6.67
com-Amazon	612307	412040	426755	556146	10.10
com-DBLP	606723	399183	426206	581805	4.28
com-Youtube	2649253	–	–	–	–

its solutions there are still worse than KaPoCE’s.

Certificates. Table 6 is the main result of this section. For these bounds CertiFlip exported its certificate, and every bound in the table is the value computed by the independent checker of Section 4.3, which accepted all eleven certificates (the check took 1 to 34s). On the eleven graphs whose support fits into memory, the CertiFlip bound certifies the best known solutions within 0.25% to 10.1% of optimal. The median gap is 6.4%, and on the collaboration networks **ca-GrQc** and **ca-HepTh** it is below 1%. The classical scalable bounds, a maximal packing of bad triangles (the bound of MatchFlipPivot) and the fractional triangle packing, certify only factors of 1.17 to 1.81 on the same graphs. As far as we know, no certificate of this quality was previously available for correlation clustering instances of this size. The certificates for **com-Amazon** and **com-DBLP** ($3 \cdot 10^5$ vertices, 10^6 edges) were computed within the 600 s budget on one core. The dense-matrix

Table 7: Ablation of the lower bound (budget 300s per variant): certified gap to the best known solution and running time. “–”: did not finish within one hour.

Bound	ca-GrQc		ca-HepTh		BitcoinAlpha+		BitcoinOTC+		ca-CondMat	
	gap (%)	s	gap (%)	s	gap (%)	s	gap (%)	s	gap (%)	s
greedy triangle packing	35.07	0	45.95	0	80.54	0	80.17	0	44.32	0
triangle packing (MWU)	28.04	0	45.26	0	86.45	1	87.62	2	43.15	3
greedy star packing	5.09	3	5.96	0	7.98	0	7.33	0	5.58	0
star packing + local search	0.73	62	0.93	46	6.34	39	5.62	55	3.23	20
block LP (cold start)	0.78	230	17.85	382	47.86	361	74.94	317	59.20	678
packing + block LP	0.22	130	0.76	304	6.16	303	5.45	468	2.04	666
packing + block LP + gap blocks	0.18	300	0.75	343	6.14	305	5.46	470	2.03	667
KaPoCE root p3 packing	23.19	47	38.14	374	76.72	9	–	–	–	–
KaPoCE root star packing	0.58	31	0.92	277	3.61	850	–	–	–	–

implementation of the KaPoCE packing bounds cannot represent these graphs.

5.4 Ablations

Components of the bound. Table 7 evaluates the components of the lower bound on five graphs whose size allows running the dense-matrix KaPoCE bounds for comparison. Each variant had a budget of 300s. Gaps are measured against the best solution of Table 5. These are the floating-point values of the solver from local runs; the checked bounds of Table 6 come from separate runs.

- *Triangle packings.* Packings of bad triangles, greedy or fractional, leave gaps of 28% to 88%. This is the factor-two regime of previous scalable bounds.
- *Star packings.* A greedy star packing on the sparse support already reaches gaps of 5% to 8% in well under a second. The ruin-and-recreate local search lowers them to 0.7% to 6.3%.
- *Block LPs.* Without a warm start, block LPs converge slowly and reach only an 18% to 75% gap within the budget; **ca-GrQc** is the exception. Started from the packing, they improve the bound in every case, to 0.2% to 6.2%. Gap-guided blocks add a little more on four of the five graphs.
- *Comparison with KaPoCE.* The root star-packing bound of the KaPoCE branch-and-bound is weaker than ours on **ca-GrQc** (0.58% against 0.18%) and on **ca-HepTh** (0.92% against 0.75%), and stronger on **BitcoinAlpha+** (3.6% against 6.1%), where it needed 850s. On **BitcoinOTC+** and **ca-CondMat** it did not finish within one hour.

Figure 6 shows the anytime behaviour on **ca-HepTh** and **com-DBLP**. From a cold start the certified gap is still 15% and 1412% after ten minutes. With the packing warm start it is below 1% and at 3%, respectively, as soon as the packing phase ends, and it keeps decreasing afterwards.

Scaling. Table 8 reports running times on synthetic sparse instances with planted clusters of mean size 10, intra-cluster density 0.7 and two random inter-cluster edges per vertex. Building the distance-two support and one pass of Louvain-type local search take seconds even at 10^6 vertices. The iterated flipping local search grows slightly faster than linearly, from 65s at 10^5 to about 20 minutes at 10^6 vertices. One million ruin-and-recreate steps of the packing local search take less than a minute beyond the linear-time initial construction.

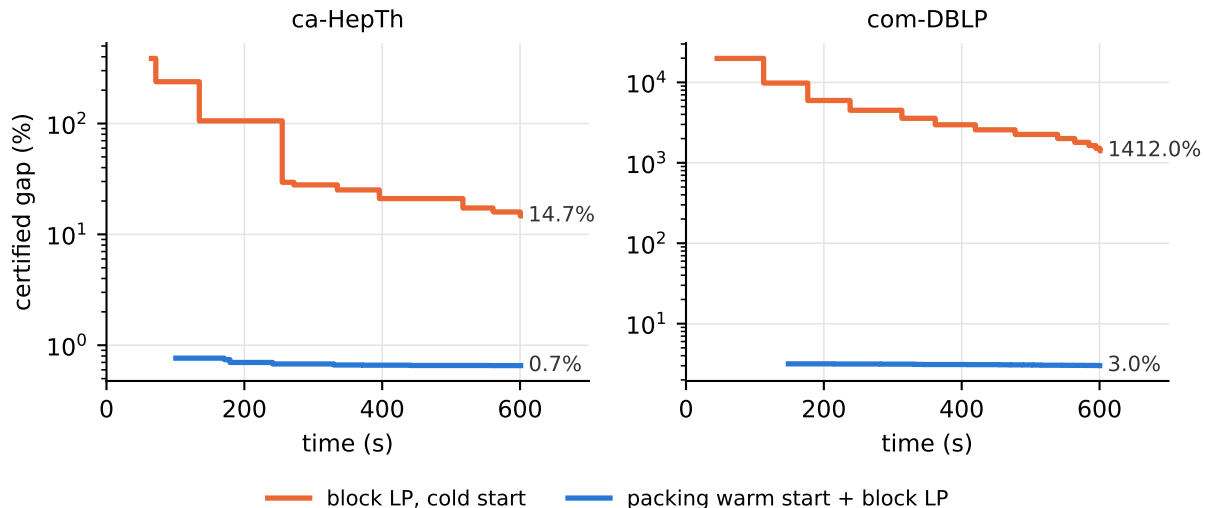


Figure 6: Certified gap $(UB - LB)/LB$ of the anytime bound over time, against the best known solution (log scale). The warm-started curve begins when the star packing has been installed.

Table 8: Running times (s) on synthetic sparse instances.

Step	$n = 10\,000$	$n = 30\,000$	$n = 100\,000$	$n = 300\,000$	$n = 1\,000\,000$
iterated flip	6.3	17.7	65.2	259.6	1209.9
louvain	0.1	0.1	0.4	1.6	6.5
star packing (1e6 iterations)	20.6	23.4	28.9	45.8	111.8
support	0.1	0.2	0.6	2.7	14.1

5.5 PXMmem against KaPoCE

Setup. The head-to-head runs used Google Colab machines with one physical core (two hyper-threads) and 12 GB of memory. On every machine the two solvers ran one after the other, each pinned to the same logical CPU while the rest of the machine was idle, with the same wall-clock budget of 600s. KaPoCE was stopped as in PACE 2021, by SIGTERM at the limit; it normally stops itself after 590s. Its seed, fixed in the released code, was set to the run seed (a one-line change), so that its ten runs per graph are independent. Its output was checked: an edit set is valid iff the edited graph is a disjoint union of cliques. A run in which KaPoCE returned no valid solution would count as a win for PXMmem, as in the PACE rules; this did not occur in any of the 150 runs. PXMmem ran with the released configuration. The Numba kernels were compiled once per machine before the timed runs, as KaPoCE is compiled ahead of time. We used all fifteen SNAP graphs, ten seeds each. The configuration of PXMmem was developed on eleven of these graphs; the PACE heuristic-track instances below were not used during development. We report the mean paired difference $\bar{\Delta}$ (PXMmem minus KaPoCE, in disagreements) with a 95% bootstrap interval, wins, ties and losses, the two-sided sign test and its Holm correction over the fifteen graphs.

Solution quality at 600s. Table 9 shows that on the four smallest graphs and on **ca-HepPh** both solvers find the same solutions in every run; PXMmem reaches them after one to three minutes. PXMmem is better in every run on **ca-AstroPh** (by 63 disagreements on average) and in eight to nine of ten runs on **ca-CondMat**, **email-Enron** and **com-DBLP** (by 11 to 34). These four differences

Table 9: PXMmem against KaPoCE, 600s per run, run one after the other on the same machine and CPU (ten seeds per graph). Mean costs; $\bar{\Delta}$: mean paired difference PXMmem–KaPoCE with a 95% bootstrap interval; W/T/L: wins, ties and losses of PXMmem; p : two-sided sign test; p_{Holm} : Holm correction over the fifteen graphs (bold if below 0.05); $t_{=}$: median time at which PXMmem first reached the final cost of KaPoCE (“–” if it did not in most runs). KaPoCE returned a valid solution in every run.

graph	n	PXMmem	KaPoCE	$\bar{\Delta}$ [95% CI]	W/T/L	p	p_{Holm}	$t_{=}$ (s)
ca-GrQc	5,242	6,050.0	6,050.0	+0.0	0/10/0	1.00	1.00	69
Bitcoin-Alpha ⁺	3,683	11,437.0	11,437.0	+0.0	0/10/0	1.00	1.00	71
Bitcoin-OTC ⁺	5,573	16,462.0	16,462.0	+0.0	0/10/0	1.00	1.00	181
ca-HepTh	9,877	15,813.0	15,813.5	−0.5 [−0.8, −0.2]	5/5/0	0.06	0.62	125
ca-HepPh	12,008	46,133.0	46,133.0	+0.0	0/10/0	1.00	1.00	107
ca-AstroPh	18,772	125,543.9	125,606.8	−62.9 [−78.1, −49.5]	10/0/0	0.002	0.029	62
ca-CondMat	23,133	56,834.3	56,845.6	−11.3 [−16.4, −6.0]	8/2/0	0.008	0.10	56
email-Enron	36,692	150,428.2	150,460.8	−32.6 [−47.5, −17.1]	9/0/1	0.021	0.26	139
loc-Brightkite	58,228	174,700.1	174,703.2	−3.1 [−6.0, −0.4]	7/0/3	0.34	1.00	432
Slashdot ⁺	75,144	351,865.0	351,869.1	−4.1 [−8.7, +0.1]	6/2/2	0.29	1.00	579
soc-Epinions	75,879	365,164.5	365,167.6	−3.1 [−22.7, +16.1]	4/0/6	0.75	1.00	–
Epinions ⁺	114,467	521,817.3	521,818.1	−0.8 [−17.6, +16.6]	5/0/5	1.00	1.00	–
com-DBLP	317,080	606,753.1	606,787.2	−34.1 [−56.5, −15.1]	9/0/1	0.021	0.26	296
com-Amazon	334,863	612,328.7	612,311.6	+17.1 [+14.5, +20.0]	0/0/10	0.002	0.029	–
com-Youtube	1,134,890	2,650,090.2	2,649,791.7	+298.5 [+40.0, +574.9]	4/0/6	0.75	1.00	–

are significant by the sign test, but after the Holm correction over fifteen graphs only **ca-AstroPh** remains so. On the four social networks of medium size the two solvers are statistically indistinguishable. KaPoCE is better in every run on **com-Amazon** (by 17 disagreements, 0.003%), which survives the correction, and on average on **com-Youtube**, where the runs of both solvers vary by several hundred disagreements and the sign test is not significant. Over all 150 runs PXMmem wins 67, ties 49 and loses 34. The differences are small in relative terms (at most 0.05%), far below the certified gaps of Section 5.3; they matter only when the last fraction of a percent is wanted. Figure 7 shows how the runs unfold on four of the graphs: PXMmem passes the final KaPoCE value within five minutes on **ca-AstroPh**, **email-Enron** and **com-DBLP**, and is still converging on **com-Amazon** when the budget ends. On **com-Amazon** we compared the two solutions of a local 300s run block by block with Theorem 11. They differed in more than 16,000 blocks, almost all ties, and KaPoCE was better in 24 blocks of 8 to 77 vertices, each containing a vertex of degree 22 to 142. Solving these blocks exactly with the MIP of Section 3.3 recovered the value of KaPoCE in 23 of them, mostly in less than a second, so the difficulty lies in finding the blocks, not in solving them.

Dependence on the budget. Figure 8 and Table 10 show the main limitation of PXMmem. At 60s and 150s KaPoCE is better overall (PXMmem wins 43 and 41 runs and loses 84 and 71; sign test $p < 0.01$ for both budgets), while at 600s the balance reverses (67 wins, 34 losses, $p = 0.001$). The short-budget losses are concentrated on the six largest graphs, where KaPoCE wins at least eight of ten runs at 60s. There the initial population (Pivot, flipping and annealing runs) takes a large part of a short budget; on **com-Youtube** PXMmem is 0.64% behind at 60s and 0.19% at 150s. On **ca-HepTh** and **ca-CondMat** PXMmem is better at every budget. In two of the ten 60s runs on **com-Youtube** KaPoCE was stopped before its local search had improved the initial solution and returned a solution 13% worse; its output was valid, so these runs count as ordinary wins, and the median is not affected by them.

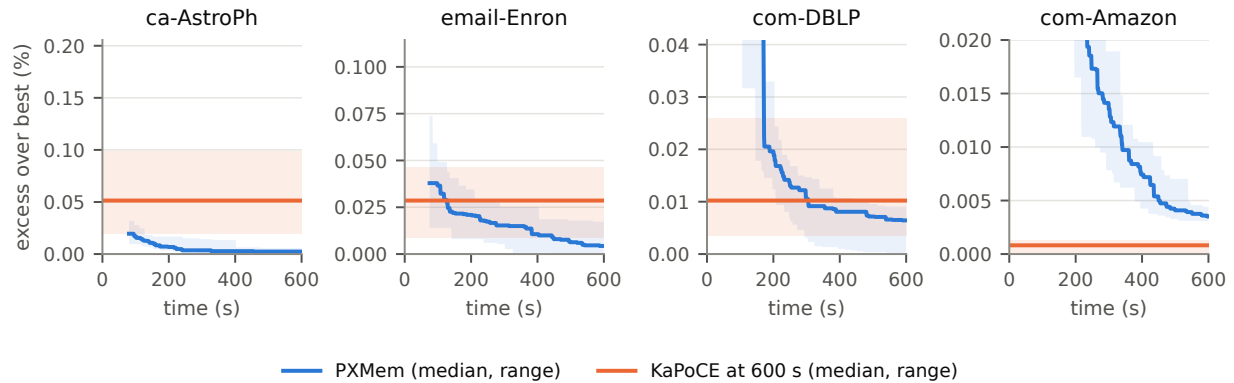


Figure 7: Best cost of PXMmem over time in the sequential 600s runs of Table 9 (median and range over ten seeds, blue), against the final cost of KaPoCE in the same runs (median and range, orange), as excess over the best value found by either solver. The PXMmem curves start when the initial population is complete.

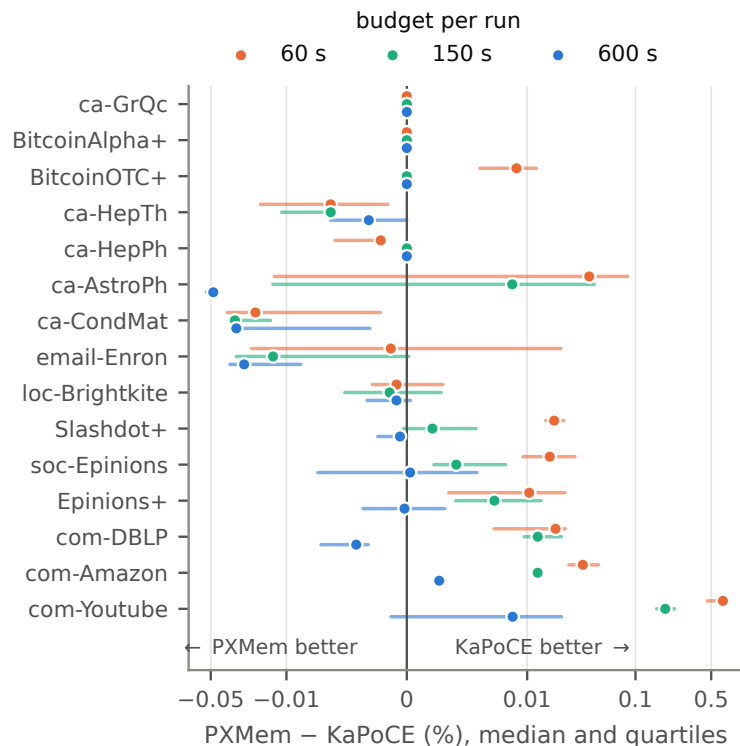


Figure 8: PXMmem against KaPoCE at three budgets, same protocol as Table 9 (ten seeds per graph and budget): median paired relative difference with quartiles (symmetric log scale; left of zero favours PXMmem). Graphs are ordered by size. Per-graph wins, ties and losses are in Table 10.

Table 10: PXMem against KaPoCE at three budgets (the data of Figure 8): median paired relative difference PXMem–KaPoCE in percent and wins, ties and losses of PXMem.

graph	60 s		150 s		600 s	
	med. (%)	W/T/L	med. (%)	W/T/L	med. (%)	W/T/L
ca-GrQc	+0.000	0/10/0	+0.000	0/10/0	+0.000	0/10/0
Bitcoin-Alpha ⁺	+0.000	2/6/2	+0.000	0/8/2	+0.000	0/10/0
Bitcoin-OTC ⁺	+0.009	0/2/8	+0.000	0/9/1	+0.000	0/10/0
ca-HepTh	−0.006	7/2/1	−0.006	8/2/0	−0.003	5/5/0
ca-HepPh	−0.002	8/2/0	+0.000	2/8/0	+0.000	0/10/0
ca-AstroPh	+0.037	3/0/7	+0.009	4/0/6	−0.047	10/0/0
ca-CondMat	−0.019	7/1/2	−0.030	9/0/1	−0.029	8/2/0
email-Enron	−0.001	5/0/5	−0.013	7/0/3	−0.025	9/0/1
loc-Brightkite	−0.001	5/0/5	−0.001	5/1/4	−0.001	7/0/3
Slashdot ⁺	+0.018	0/0/10	+0.002	3/0/7	−0.001	6/2/2
soc-Epinions	+0.016	1/0/9	+0.004	2/0/8	+0.000	4/0/6
Epinions ⁺	+0.010	1/0/9	+0.007	1/0/9	+0.000	5/0/5
com-DBLP	+0.018	1/0/9	+0.013	0/0/10	−0.004	9/0/1
com-Amazon	+0.033	1/0/9	+0.012	0/0/10	+0.003	0/0/10
com-Youtube	+0.641	2/0/8	+0.188	0/0/10	+0.009	4/0/6

Table 11: PACE 2021 heuristic track (200 instances, one run of 600 s per solver and instance, same protocol as Table 9), grouped by the number of vertices: wins, ties and losses of PXMem, the summed difference of the costs and its value relative to the summed KaPoCE cost.

vertices	instances	W/T/L	$\sum$ PXMem–KaPoCE	rel. (%)
$< 10^3$	93	0/92/1	+5	+0.0006
10^3 – 10^4	80	8/68/4	+80	+0.0005
10^4 – 10^5	13	7/4/2	−236	−0.0063
$\geq 10^5$	14	6/0/8	+7122	+0.0425
all	200	21/164/15	+6971	+0.0194

PACE 2021 heuristic track. Table 11 and Figure 9 report one run of each solver on each of the 200 instances, with the protocol and budget of Table 9. KaPoCE returned a valid solution on every instance. Most instances are easy for both solvers: below 10^4 vertices they find the same solution on all but 13 of 173 instances. Between 10^4 and $5 \cdot 10^5$ vertices PXMem is better on 13 of 23 instances and worse on 6; its largest gain is 835 disagreements (**heur190**, 0.05%). On the four instances with more than 10^6 vertices (up to $2.0 \cdot 10^6$) it is behind by 0.02% to 0.32%, as on **com-Youtube**: at this size 600 s is too short for its population. These four instances dominate the summed difference in Table 11. Over all instances PXMem wins 21, ties 164 and loses 15; the sign test does not separate the solvers ($p = 0.41$).

Parallel runs and post-processing. Theorem 11 gives a way to use several cores that heuristics without an exact recombination do not have. Table 12 reports four independent PXMem runs of 300 s that ran at the same time on four cores of our 4-core machine, compared with the best of k

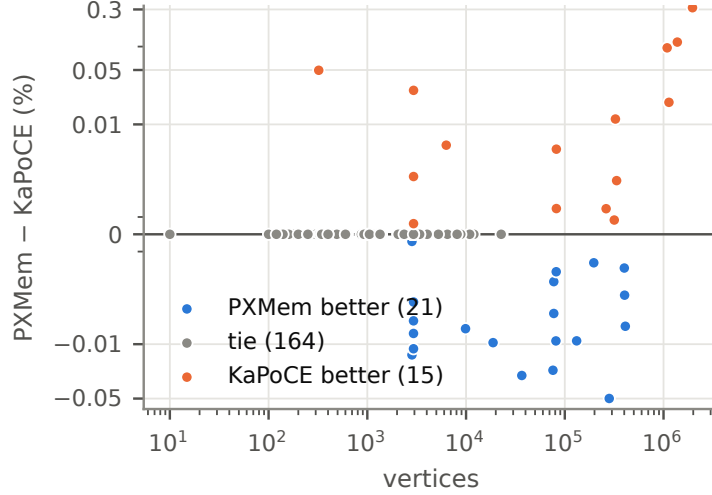


Figure 9: PACE 2021 heuristic track: relative difference PXMmem–KaPoCE per instance against the number of vertices (one 600s run per solver; symmetric log scale). Grey points on the zero line are ties.

Table 12: Four independent PXMmem runs (300s each, run in parallel): best of k runs and partition crossover of k runs (averaged over all subsets of size k), KaPoCE (300s, one core), and the partition crossover of the KaPoCE solution with one PXMmem solution (averaged over the four runs) and with all four. The best value in each row is bold.

graph	1 run	2 runs		4 runs		KaPoCE	PX with KaPoCE	
		best	PX	best	PX		1 run	4 runs
Bitcoin-OTC ⁺	16,462	16,462	16,462	16,462	16,462	16,462	16,462	16,462
ca-AstroPh	125,544.5	125,542.8	125,541.7	125,542	125,540	125,567	125,542.8	125,540
email-Enron	150,453.8	150,445.8	150,444.5	150,433	150,432	150,438	150,435.8	150,432
loc-Brightkite	174,706.2	174,704.5	174,699.7	174,703	174,697	174,699	174,696	174,695
com-DBLP	606,773.8	606,761.2	606,735.3	606,754	606,720	606,779	606,728.2	606,719
com-Amazon	612,374.5	612,361.7	612,330.3	612,350	612,317	612,316	612,309.2	612,308

of them and with their partition crossover, and with KaPoCE on one core. Combining two runs by partition crossover is better than the best of four runs on four of the five graphs where the runs differ (all but **email-Enron**), and never worse than the best of two. With four runs PXMmem matches or beats KaPoCE on five of the six graphs and is one disagreement behind on **com-Amazon**. Applied to the solutions of KaPoCE, the crossover with one PXMmem run improves them on all five graphs where they are not already equal to ours (by 2 to 51 disagreements). Crossing the KaPoCE solution with all four PXMmem runs gives the best value in every row; on **loc-Brightkite**, **com-DBLP** and **com-Amazon** it is better than every input, including the crossover of the four PXMmem runs (by 2, 1 and 8 disagreements). This makes the crossover useful as a post-processing step for any solver. These numbers compare four cores with one and complement, but do not replace, the single-core comparison of Table 9.

Memory. PXMmem needs more memory than KaPoCE. The peak resident set sizes measured in the runs of Table 9 are 360 MB to 370 MB against 180 MB to 190 MB on **com-Amazon** and **com-DBLP**,

and 970 MB against 730 MB on `com-Youtube`.

6 Conclusion

Correlation clustering has moved quickly in theory, but certified solution quality on real instances had not kept pace. Three ingredients make LP-based certificates usable at scale: the distance-two relaxation, the anytime block-dual bound warm-started by sparse packings, and the gap decomposition. They also give a primal method that knows where to search. Our heuristic version of the iterated flipping local search of Cohen-Addad et al. [25] comes within a few tenths of a percent of the best engineered heuristic in a fraction of its running time. The exact partition crossover closes the remaining gap on most graphs: PXMem keeps the a-priori guarantee, is not significantly worse than KaPoCE on fourteen of fifteen graphs at equal budget on the same core, and turns additional cores into better solutions. The main combinatorial arguments are machine-checked, and every reported lower bound is confirmed by an independent exact checker.

Limitations.

- KaPoCE remains better than PXMem on `com-Amazon` (by 0.003% at 600s) and, not significantly, on `com-Youtube`. At short budgets of 60s and 150s it is better overall, most clearly on the six largest graphs (median 0.002% to 0.64%), where our initial population takes too much of the budget. The same holds at 600s on the four PACE heuristic-track instances with more than 10^6 vertices (0.02% to 0.32%). PXMem also needs up to twice the memory of KaPoCE.
- The formal development covers the statements listed in Table 1, not all results of the paper, and it covers the mathematics, not the code. The Pivot guarantee used in Theorems 8 and 12 enters it as a hypothesis.
- On dense instances the packing bounds of the KaPoCE branch-and-bound remain stronger than ours.
- The distance-two support has $O(\sum_v \deg(v)^2)$ pairs. On graphs with strong hubs it exceeds the memory of our machine, and CertiFlip then reports no bound. Restricting the support to centres of bounded degree keeps the bound valid and is a natural next step.
- The 2.06 a-priori guarantee applies only when a single block covers the graph and separation converges.

Open directions. Parallel block solves; cluster-LP columns inside blocks, where the recent roundings of Cao et al. [16] and García-Soriano and Schohn [35] could be applied; and overlapping blocks with Lagrangian cost splitting.

Code and data availability. All code, instance lists and raw results are available at https://github.com/vinhqdang/correlation_clustering.

References

- [1] Ahmed Abbas and Paul Swoboda. RAMA: A rapid multicut algorithm on GPU. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022)*, pages 8183–8192, 2022. doi: 10.1109/CVPR52688.2022.00802.

- [2] Nir Ailon, Moses Charikar, and Alantha Newman. Aggregating inconsistent information: Ranking and clustering. *Journal of the ACM*, 55(5):23:1–23:27, 2008. doi: 10.1145/1411509.1411513. Conference version: STOC 2005, pp. 684–693, doi:10.1145/1060590.1060692.
- [3] David Applegate, Robert Bixby, Vašek Chvátal, and William Cook. TSP cuts which do not conform to the template paradigm. In Michael Jünger and Denis Naddef, editors, *Computational Combinatorial Optimization: Optimal or Provably Near-Optimal Solutions*, volume 2241 of *LNCS*, pages 261–303. Springer, 2001. doi: 10.1007/3-540-45586-8_7.
- [4] Nikhil Bansal, Avrim Blum, and Shuchi Chawla. Correlation clustering. *Machine Learning*, 56(1-3):89–113, 2004. doi: 10.1023/B:MACH.0000033116.57574.95. Conference version: FOCS 2002, pp. 238–247, doi:10.1109/SFCS.2002.1181947.
- [5] Lucas Bastos, Luiz Satoru Ochi, Fábio Protti, Anand Subramanian, Ivan César Martins, and Rian Gabriel S. Pinheiro. Efficient algorithms for cluster editing. *Journal of Combinatorial Optimization*, 31(1):347–371, 2016. doi: 10.1007/s10878-014-9756-7.
- [6] Soheil Behnezhad, Moses Charikar, Weiyun Ma, and Li-Yang Tan. Almost 3-approximate correlation clustering in constant rounds. In *IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 720–731, 2022. doi: 10.1109/FOCS54457.2022.00074.
- [7] Una Benlic and Jin-Kao Hao. A multilevel memetic approach for improving graph k-partitions. *IEEE Transactions on Evolutionary Computation*, 15(5):624–642, 2011. doi: 10.1109/TEVC.2011.2136346.
- [8] Sonja Biedermann, Monika Henzinger, Christian Schulz, and Bernhard Schuster. Memetic graph clustering. In *17th International Symposium on Experimental Algorithms (SEA 2018)*, volume 103 of *LIPIcs*, pages 3:1–3:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.SEA.2018.3.
- [9] Thomas Bläsius, Philipp Fischbeck, Lars Gottesbüren, Michael Hamann, Tobias Heuer, Jonas Spinner, Christopher Weyand, and Marcus Wilhelm. PACE solver description: KaPoCE: A heuristic cluster editing algorithm. In *16th International Symposium on Parameterized and Exact Computation (IPEC 2021)*, volume 214 of *LIPIcs*, pages 31:1–31:4, 2021. doi: 10.4230/LIPIcs.IPEC.2021.31.
- [10] Thomas Bläsius, Philipp Fischbeck, Lars Gottesbüren, Michael Hamann, Tobias Heuer, Jonas Spinner, Christopher Weyand, and Marcus Wilhelm. A branch-and-bound algorithm for cluster editing. In *20th International Symposium on Experimental Algorithms (SEA 2022)*, volume 233 of *LIPIcs*, pages 13:1–13:19, 2022. doi: 10.4230/LIPIcs.SEA.2022.13.
- [11] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008. doi: 10.1088/1742-5468/2008/10/P10008.
- [12] Sebastian Böcker. A golden ratio parameterized algorithm for cluster editing. *Journal of Discrete Algorithms*, 16:79–89, 2012. doi: 10.1016/j.jda.2012.04.005. Conference version: IWOCA 2011, doi:10.1007/978-3-642-25011-8_7.
- [13] Sebastian Böcker, Sebastian Briesemeister, Quang Bao Anh Bui, and Anke Truss. Going weighted: Parameterized algorithms for cluster editing. *Theoretical Computer Science*, 410(52):5467–5480, 2009. doi: 10.1016/j.tcs.2009.05.006. Conference version: COCOA 2008, pp. 1–12, doi:10.1007/978-3-540-85097-7_1.

- [14] Sebastian Böcker, Sebastian Briesemeister, and Gunnar W. Klau. Exact algorithms for cluster editing: Evaluation and experiments. *Algorithmica*, 60(2):316–334, 2011. doi: 10.1007/s00453-009-9339-7.
- [15] Michael J. Brusco and Hans-Friedrich Köhn. Clustering qualitative data based on binary equivalence relations: Neighborhood search heuristics for the clique partitioning problem. *Psychometrika*, 74(4):685–703, 2009. doi: 10.1007/s11336-009-9126-z.
- [16] Nairen Cao, Vincent Cohen-Addad, Euiwoong Lee, Shi Li, Alantha Newman, and Lukas Vogl. Understanding the cluster linear program for correlation clustering. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1605–1616, 2024. doi: 10.1145/3618260.3649749. Conference version claims 1.437; arXiv v3 (2025) reports a gap in that proof and proves 1.485.
- [17] Nairen Cao, Vincent Cohen-Addad, Shi Li, Euiwoong Lee, David Rasmussen Lolck, Alantha Newman, Mikkel Thorup, Lukas Vogl, Shuyi Yan, and Hanwen Zhang. Solving the correlation cluster LP in sublinear time. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1154–1165, 2025. doi: 10.1145/3717823.3718181.
- [18] Yixin Cao and Jianer Chen. Cluster editing: Kernelization based on edge cuts. *Algorithmica*, 64(1):152–169, 2012. doi: 10.1007/s00453-011-9595-1.
- [19] Moses Charikar, Venkatesan Guruswami, and Anthony Wirth. Clustering with qualitative information. *Journal of Computer and System Sciences*, 71(3):360–383, 2005. doi: 10.1016/j.jcss.2004.10.012. Conference version: FOCS 2003, pp. 524–533, doi:10.1109/SFCS.2003.1238225.
- [20] Shuchi Chawla, Konstantin Makarychev, Tselil Schramm, and Grigory Yaroslavtsev. Near optimal LP rounding algorithm for correlation clustering on complete and complete k-partite graphs. In *Proceedings of the 47th Annual ACM Symposium on Theory of Computing (STOC)*, pages 219–228, 2015. doi: 10.1145/2746539.2746604.
- [21] Jianer Chen and Jie Meng. A 2k kernel for the cluster editing problem. *Journal of Computer and System Sciences*, 78(1):211–220, 2012. doi: 10.1016/j.jcss.2011.04.001. Conference version: COCOON 2010, doi:10.1007/978-3-642-14031-0_49.
- [22] Vincent Cohen-Addad, Silvio Lattanzi, Slobodan Mitrović, Ashkan Norouzi-Fard, Nikos Parotsidis, and Jakub Tarnawski. Correlation clustering in constant many parallel rounds. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 2069–2078, 2021.
- [23] Vincent Cohen-Addad, Euiwoong Lee, and Alantha Newman. Correlation clustering with Sherali-Adams. In *63rd IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 651–661, 2022. doi: 10.1109/FOCS54457.2022.00068.
- [24] Vincent Cohen-Addad, Euiwoong Lee, Shi Li, and Alantha Newman. Handling correlated rounding error via preclustering: A 1.73-approximation for correlation clustering. In *64th IEEE Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1082–1104, 2023. doi: 10.1109/FOCS57990.2023.00065.
- [25] Vincent Cohen-Addad, David Rasmussen Lolck, Marcin Pilipczuk, Mikkel Thorup, Shuyi Yan, and Hanwen Zhang. Combinatorial correlation clustering. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1617–1628, 2024. doi: 10.1145/3618260.3649712.

- [26] Emilie Danna, Edward Rothberg, and Claude Le Pape. Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming*, 102(1):71–90, 2005. doi: 10.1007/s10107-004-0518-7.
- [27] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.
- [28] Erik D. Demaine, Dotan Emanuel, Amos Fiat, and Nicole Immorlica. Correlation clustering in general weighted graphs. *Theoretical Computer Science*, 361(2-3):172–187, 2006. doi: 10.1016/j.tcs.2006.05.008.
- [29] Micha Elsner and Warren Schudy. Bounding and comparing methods for correlation clustering beyond ILP. In *Proceedings of the Workshop on Integer Linear Programming for Natural Language Processing*, pages 19–27. Association for Computational Linguistics, 2009. doi: 10.3115/1611638.1611641.
- [30] Anton V. Ereemeev and Julia V. Kovalenko. Optimal recombination in genetic algorithms for combinatorial optimization problems: Part I. *Yugoslav Journal of Operations Research*, 24(1): 1–20, 2014. doi: 10.2298/YJOR130731040E.
- [31] Emanuel Falkenauer. *Genetic Algorithms and Grouping Problems*. John Wiley & Sons, Chichester, UK, 1998. ISBN 978-0-471-97150-4.
- [32] Matteo Fischetti and Andrea Lodi. Local branching. *Mathematical Programming*, 98(1-3): 23–47, 2003. doi: 10.1007/s10107-003-0395-5.
- [33] Lisa K. Fleischer. Approximating fractional multicommodity flow independent of the number of commodities. *SIAM Journal on Discrete Mathematics*, 13(4):505–520, 2000. doi: 10.1137/S0895480199355754.
- [34] Philippe Galinier and Jin-Kao Hao. Hybrid evolutionary algorithms for graph coloring. *Journal of Combinatorial Optimization*, 3(4):379–397, 1999. doi: 10.1023/A:1009823419804.
- [35] David García-Soriano and Antoine Schohn. Approximate dual separation for the cluster LP: a 1.387 approximation for correlation clustering. arXiv preprint arXiv:2607.27829, 2026.
- [36] Naveen Garg and Jochen Könemann. Faster and simpler algorithms for multicommodity flow and other fractional packing problems. *SIAM Journal on Computing*, 37(2):630–652, 2007. doi: 10.1137/S0097539704446232.
- [37] Martin Grötschel and Yoshiko Wakabayashi. A cutting plane algorithm for a clustering problem. *Mathematical Programming*, 45(1-3):59–96, 1989. doi: 10.1007/BF01589097.
- [38] Martin Grötschel and Yoshiko Wakabayashi. Facets of the clique partitioning polytope. *Mathematical Programming*, 47(1-3):367–387, 1990. doi: 10.1007/BF01580870.
- [39] Jiong Guo. A more effective linear kernelization for cluster editing. *Theoretical Computer Science*, 410(8-10):718–726, 2009. doi: 10.1016/j.tcs.2008.10.021.
- [40] Felix Hausberger, Marcelo Fonseca Faraj, and Christian Schulz. Scalable multilevel and memetic signed graph clustering. In *Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*, pages 81–94, 2025. doi: 10.1137/1.9781611978339.7.

- [41] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.
- [42] Jannik Irmay, Lucas Fabian Naumann, and Bjoern Andres. Chorded cycle facets of the clique partitioning polytope. arXiv preprint arXiv:2411.03407, 2024.
- [43] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998. doi: 10.1137/S1064827595287997.
- [44] Leon Kellerhals, Tomohiro Koana, André Nichterlein, and Philipp Zschoche. The PACE 2021 parameterized algorithms and computational experiments challenge: Cluster editing. In Petr A. Golovach and Meirav Zehavi, editors, *16th International Symposium on Parameterized and Exact Computation (IPEC 2021)*, volume 214 of *LIPIcs*, pages 26:1–26:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi: 10.4230/LIPIcs.IPEC.2021.26.
- [45] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.
- [46] Miyuki Koshimura, Emi Watanabe, Yuko Sakurai, and Makoto Yokoo. Concise integer linear programming formulation for clique partitioning problems. *Constraints*, 27(1-2):99–115, 2022. doi: 10.1007/s10601-022-09326-z.
- [47] Srijan Kumar, Francesca Spezzano, V. S. Subrahmanian, and Christos Faloutsos. Edge weight prediction in weighted signed networks. In *IEEE 16th International Conference on Data Mining (ICDM)*, pages 221–230, 2016. doi: 10.1109/ICDM.2016.0033.
- [48] Jan-Hendrik Lange, Andreas Karrenbauer, and Bjoern Andres. Partial optimality and fast lower bounds for weighted correlation clustering. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 2892–2901, 2018.
- [49] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Signed networks in social media. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 1361–1370, 2010. doi: 10.1145/1753326.1753532.
- [50] Evgeny Levinkov, Alexander Kirillov, and Bjoern Andres. A comparative study of local search algorithms for correlation clustering. In *German Conference on Pattern Recognition (GCPR)*, volume 10496 of *LNCS*, pages 103–114, 2017. doi: 10.1007/978-3-319-66709-6_9.
- [51] Zhi Lu, Yi Zhou, and Jin-Kao Hao. A hybrid evolutionary algorithm for the clique partitioning problem. *IEEE Transactions on Cybernetics*, 52(9):9391–9403, 2022. doi: 10.1109/TCYB.2021.3051243.
- [52] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi: 10.1016/j.cosrev.2010.09.009.
- [53] Atsushi Miyauchi and Noriyoshi Sukegawa. Redundant constraints in the standard formulation for the clique partitioning problem. *Optimization Letters*, 9(1):199–207, 2015. doi: 10.1007/s11590-014-0754-6.
- [54] Atsushi Miyauchi, Tomohiro Sonobe, and Noriyoshi Sukegawa. Exact clustering via integer programming and maximum satisfiability. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11519.

- [55] Michael Ovelgönne and Andreas Geyer-Schulz. An ensemble learning strategy for graph clustering. In David A. Bader, Henning Meyerhenke, Peter Sanders, and Dorothea Wagner, editors, *Graph Partitioning and Graph Clustering*, volume 588 of *Contemporary Mathematics*, pages 187–205. American Mathematical Society, 2013. doi: 10.1090/conm/588/11701.
- [56] Fábio Protti, Maise Dantas da Silva, and Jayme Luiz Szwarcfiter. Applying modular decomposition to parameterized cluster editing problems. *Theory of Computing Systems*, 44(1):91–104, 2009. doi: 10.1007/s00224-007-9032-7.
- [57] Cameron Ruggles, Nate Veldt, and David F. Gleich. A parallel projection method for metric constrained optimization. In *Proceedings of the SIAM Workshop on Combinatorial Scientific Computing (CSC)*, pages 43–53, 2020. doi: 10.1137/1.9781611976229.5.
- [58] Peter Sanders and Christian Schulz. Distributed evolutionary graph partitioning. In *Proceedings of the 14th Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 16–29. SIAM, 2012. doi: 10.1137/1.9781611972924.2.
- [59] Paul Shaw. Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming – CP98*, volume 1520 of *LNCS*, pages 417–431, 1998. doi: 10.1007/3-540-49481-2_30.
- [60] Paul Swoboda and Bjoern Andres. A message passing algorithm for the minimum cost multicut problem. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017)*, pages 4990–4999, 2017. doi: 10.1109/CVPR.2017.530.
- [61] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. doi: 10.1145/3372885.3373824.
- [62] Renato Tinós, Darrell Whitley, and Francisco Chicano. Partition crossover for pseudo-boolean optimization. In *Proceedings of the 2015 ACM Conference on Foundations of Genetic Algorithms XIII (FOGA)*, pages 137–149. ACM, 2015. doi: 10.1145/2725494.2725497.
- [63] Renato Tinós, Darrell Whitley, and Gabriela Ochoa. A new generalized partition crossover for the traveling salesman problem: Tunneling between local optima. *Evolutionary Computation*, 28(2):255–288, 2020. doi: 10.1162/evco_a_00254.
- [64] V. A. Traag, L. Waltman, and N. J. van Eck. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9:5233, 2019. doi: 10.1038/s41598-019-41695-z.
- [65] Nate Veldt. Correlation clustering via strong triadic closure labeling: Fast approximation algorithms and practical lower bounds. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, volume 162 of *PMLR*, pages 22060–22083, 2022.
- [66] Nate Veldt, David F. Gleich, and Anthony Wirth. A correlation clustering framework for community detection. In *Proceedings of the 2018 World Wide Web Conference (WWW)*, pages 439–448, 2018. doi: 10.1145/3178876.3186110.
- [67] Nate Veldt, David F. Gleich, Anthony Wirth, and James Saunderson. Metric-constrained optimization for graph clustering algorithms. *SIAM Journal on Mathematics of Data Science*, 1(2):333–355, 2019. doi: 10.1137/18M1217152.

- [68] Darrell Whitley, Doug Hains, and Adele Howe. Tunneling between optima: Partition crossover for the traveling salesman problem. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, pages 915–922, 2009. doi: 10.1145/1569901.1570026.
- [69] Darrell Whitley, Doug Hains, and Adele Howe. A hybrid genetic algorithm for the traveling salesman problem using generalized partition crossover. In *Parallel Problem Solving from Nature – PPSN XI*, volume 6238 of *LNCIS*, pages 566–575. Springer, 2010. doi: 10.1007/978-3-642-15844-5_57.
- [70] Tobias Wittkop, Jan Baumbach, Francisco P. Lobo, and Sven Rahmann. Large scale clustering of protein sequences with FORCE – A layout based heuristic for weighted cluster editing. *BMC Bioinformatics*, 8:396, 2007. doi: 10.1186/1471-2105-8-396.
- [71] Tobias Wittkop, Dorothea Emig, Sita Lange, Sven Rahmann, Mario Albrecht, John H. Morris, Sebastian Böcker, Jens Stoye, and Jan Baumbach. Partitioning biological data with transitivity clustering. *Nature Methods*, 7(6):419–420, 2010. doi: 10.1038/nmeth0610-419.
- [72] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth. In *IEEE 12th International Conference on Data Mining (ICDM)*, pages 745–754, 2012. doi: 10.1109/ICDM.2012.138.
- [73] Yi Zhou, Jin-Kao Hao, and Adrien Goëffon. A three-phased local search approach for the clique partitioning problem. *Journal of Combinatorial Optimization*, 32(2):469–491, 2016. doi: 10.1007/s10878-015-9964-9.